



Bachelor Thesis

Comparing Heuristics with Exact Approaches for the Densest Subgraph Problem

Martin Schlaier

30th September 2025

Summer Semester

First Examiner:

Dr. Jonas Sauer

Second Examiner:

Professor Dr. Heiko Röglin

Department of Computer Science
University of Bonn

Acknowledgements

I would like to express my sincere gratitude to my supervisor, **Dr. Jonas Sauer**, for his invaluable guidance, constructive criticism, and unwavering support throughout the entire process of writing this thesis. His insights were crucial to the completion of this work.

I am also thankful to **Alexis Pott**, **Tommy Beck**, and my father **Winfried Schlaier** for their meticulous proofreading and helpful suggestions, which significantly improved the quality of the final manuscript.

Finally, I want to thank my parents for their continuous encouragement and support. A special thank you also goes to my friends, who made the entire university experience enjoyable and kept me sane throughout this process.

Declaration of generative AI and AI-assited technologies in the writing process

I hereby declare that generative artificial intelligence tools in the form of **ChatGPT-4o**, **ChatGPT-5 mini**, **Gemini 2.5 Flash**, **Gemini 2.5 Pro** and **Claude Sonnet 4** were used in the writing process. These tools were solely used to improve language and readability of this work. I edited and reviewed all the generated and modified content as needed and take full responsibility for the content in this work.

Abstract

The Densest Subgraph Problem (DSP) seeks to find a subgraph with the highest ratio between edges and vertices, a fundamental challenge with applications in community detection, fraud analysis, and bioinformatics. While polynomial-time exact algorithms exist via static maximum flow reductions, approximation methods like the iterative peeling algorithm, **Greedy++**, have long been favored in practice due to their perceived speed. This thesis challenges this prevailing view by providing a comprehensive empirical evaluation of modern exact algorithms against established approximation methods.

We implement and evaluate several algorithmic approaches: two novel exact algorithms (**DSP-FBA** and **DSP-Intersect**), the **Incremental Parametric Cut (IPC)**, and the **Greedy++** approximation algorithm. Our experimental study uses a diverse set of 47 real-world graphs, including 27 unweighted and 20 weighted instances from social networks, web graphs, and biological datasets.

Our key findings demonstrate that exact algorithms can be highly competitive with their approximation counterparts. They consistently achieve optimal solutions with runtimes that are only up to 3.1 times longer than Greedy++’s 3-iteration version, which offers a reliable 90% approximation quality. We find that performance of the exact algorithms is critically dependent on graph characteristics, particularly degree variance. **DSP-FBA** with parametric pseudo flow excels on low-variance graphs, while **IPC** performs best on high-variance instances.

We also evaluate graph pruning techniques based on Charikar’s greedy algorithm. For graphs with high degree variance, pruning provides substantial benefits, achieving size reductions of up to $100\,000\times$ in vertices and $1\,300\times$ in edges. Pruning often becomes the dominant factor in the total runtime, making exact algorithms even more competitive. In fact, after pruning the exact algorithms are always faster than Greedy++ without pruning, in median more than $2\times$ faster. However, pruning is far less effective for weighted graphs, where preprocessing overhead often outweighs the computational savings.

These results offer evidence-based guidance for algorithm selection in dense subgraph discovery and suggest that modern exact algorithms deserve greater consideration, especially when solution optimality is crucial.

Contents

1. Introduction	1
1.1. Motivation	1
1.2. Related Work	1
1.3. Our Contribution	2
2. Preliminaries	5
2.1. Densest Subgraph Problem	5
2.2. Maximum Flow and Minimum Cut	5
2.3. Parametric Flow	7
2.4. Solving DSP with Reduction to Parametric Flow	9
3. Exact Algorithms	13
3.1. Goldberg’s Maximum-Flow-Based Algorithm	13
3.2. Parametric Max-Flow for DSP	13
3.2.1. Solving the DSP by finding the first breakpoint	14
3.2.2. DSP with an Intersection Scheme	15
3.2.3. Incremental Parametric Cut (IPC)	17
3.2.4. Difference IPC vs DSP-Intersect and Runtimes	18
4. Approximation Algorithms	19
4.1. Charikar Greedy-Algorithm	19
4.2. Iterative Peeling Algorithm Greedy++	20
5. Graph Pruning	23
5.1. Pruning Mechanisms: Leveraging Graph Structure	23
5.1.1. Structural Pruning via Graph Partitioning	23
5.1.2. Algorithmic Pruning via Degree Peeling	23
5.2. A Pruning-Accelerated Framework for DSP	24
6. Experimental Setup	25
6.1. Hardware	25
6.2. Implementation	25
6.3. Instances	25
6.4. Algorithm Performance Metrics	27
7. Results	29
7.1. Comparing First Breakpoint Algorithms	29
7.2. Comparing IPC and DSP-Intersect with PF and PR	30

7.3. Comparing IPC in its Versions	30
7.4. Comparing DSP-FBA FPPF and IPC	32
7.4.1. Unweighted Instances	32
7.4.2. Weighted Instances	35
7.5. Iterations performed by Greedy++	36
7.6. Comparing Greedy++ to the Exact Algorithms	36
7.6.1. Unweighted Instances	37
7.6.2. Weighted Instances	38
8. Graph Pruning	41
8.1. Size Reduction	41
8.2. Pruning Unweighted Instances	42
8.2.1. Comparing the Speedup of Each Algorithm in Comparison to Un- pruned Runtime	42
8.2.2. Ratio of Pruning Time vs Algorithm Time	43
8.2.3. Comparing the exact Algorithms on Pruned Unweighted Instances	44
8.2.4. Variant 2 of Pruning Unweighted Instances	45
8.2.5. Final Comparison on Unweighted Instances: Greedy++ vs Exact Algorithms	47
8.3. Pruning on Weighted Instances	48
9. Conclusion	51
A. Absolute Runtimes	53
Bibliography	57

1. Introduction

1.1. Motivation

Identifying highly interconnected structures within large networks is a fundamental problem in graph analytics. The **Densest Subgraph Problem (DSP)** formalizes this challenge by seeking a subgraph with the highest ratio of edges to vertices, thereby revealing the most cohesive subsets within a graph (Picard et al. 1982; Goldberg 1984).

Solving the DSP is critical for a wide range of real-world applications. Its applications range from community detection in social networks and fraud detection on financial or online platforms to the identification of protein complexes in bioinformatics and the analysis of botnets in cybersecurity (Lanciano et al. 2024).

Exact algorithms for DSP have traditionally relied on reducing the problem to multiple calls of the **static maximum flow problem** (Picard et al. 1982; Goldberg 1984). Although these methods guarantee optimal solutions, their runtime often becomes prohibitive on very large graphs. Consequently, approximation methods such as the widely used **greedy peeling algorithm** are employed, trading optimality for efficiency (Charikar 2000).

Recent advances in parametric maximum flow have made it possible to apply exact algorithms to large-scale graphs. The DSP can be formulated as a problem of finding the optimal value of a parameter (the density). The approach taken by Goldberg (1984) is to test different values for this parameter using a binary search, but this requires a complete and computationally intensive maximum flow calculation at each step.

Recent parametric flow algorithms make it possible to circumvent these redundant computations by efficiently updating an existing flow solution for a slightly modified parameter.

This thesis presents two new exact algorithms based on these principles and evaluates them against leading approximation and exact methods. The results show that if high solution quality is critical, modern exact techniques can match or even surpass approximation approaches. Furthermore, we propose a hybrid strategy that prunes the graph before applying the computationally intensive flow algorithms.

1.2. Related Work

The development of polynomial-time exact algorithms for DSP has been a significant area of research. Picard et al. (1982) published the first exact algorithm, which was fundamentally built on maximum-flow computations. This was later advanced by Goldberg (1984), who proposed a version with improved time complexity. A different approach was

taken by Charikar (2000), who designed an exact algorithm based on a linear program (LP). The existence of an LP-based algorithm is not surprising, as the max-flow problem can be formulated as an LP. However, Charikar’s method offers a new perspective, allowing his greedy-peeling approximation algorithm to be understood as a primal-dual algorithm for DSP. This greedy algorithm, which iteratively removes nodes with the lowest degree, provides a 2-approximation.

Building upon this concept, **iterative peeling algorithms** have been developed. Instead of performing the peeling process once using node degree as the criterion, it is executed multiple times. In each new iteration, more complex node weights, influenced by the results of the previous iterations, are used instead of the simple degree. Boob et al. (2020) introduced the first algorithm of this type, called *Greedy++*, and demonstrated that it achieves a near-optimal result in just a few iterations and is an order of magnitude faster than Goldberg’s maximum flow algorithm. Chekuri et al. (2022) showed that Greedy++ is an approximation scheme that can achieve a value arbitrarily close to the optimum given enough iterations, although the required number of iterations can be extremely high in theory.

Hochbaum (2024) introduced the *Incremental Parametric Cut* procedure (IPC), which uses the pseudoflow algorithm Hochbaum (2008). This approach uses previous results for initialization when the density parameter changes monotonically, thereby avoiding wasteful recomputations. Techniques like **restartable maximum flow** allow an existing flow solution to be efficiently updated after minor changes to the graph, instead of restarting the algorithm from scratch. Hochbaum (2024) claim that their algorithm is faster by several orders of magnitude than *Greedy++*. However, the publication cited to support this claim is not available online.

Recent advances have produced new parametric flow algorithms with unique capabilities. The Parametric Breadth-First Search (PBFS) (Beines et al. 2025) and Parametric Pseudo Flow (PPF) (Sauer 2025) are the first algorithms designed to find a problem’s breakpoints—critical parameter values where the minimum cut’s structure changes—in strictly monotone order. This specific property opens up new strategies for solving ratio problems like the DSP.

The optimal density for the DSP corresponds to the breakpoint of the highest value. The existence of algorithms that find breakpoints in order enables a new solution strategy: if the problem can be reformulated such that this highest-value breakpoint becomes the *first* to be found, the solution process could be significantly accelerated.

1.3. Our Contribution

A central contribution of this work is the first systematic, empirical comparative study of modern parametric maximum-flow algorithms for the densest subgraph problem (DSP) versus established approximation methods. The investigation covers 47 real-world graphs and includes the first comprehensive and publicly documented evaluation of the Incremental Parametric Cut procedure (IPC) by Hochbaum (2024), given that their own empirical study is not publicly available. To enable this evaluation we implement and

assess two novel exact algorithms:

DSP-FBA, which leverages recent monotone parametric flow algorithms to find the solution directly by identifying the first breakpoint in a reformulated network, making it the only one required.

DSP-Intersect, an intersection-based algorithm using restartable maximum flow with both push-relabel and pseudoflow implementations, a similar approach to IPC.

Our experiments reveal that the choice of the best algorithm strongly depends on graph properties, particularly degree variance. For graphs with low variance, **DSP-FBA** performs best, whereas for graphs with high variance, IPC is superior. Confirming the hypothesis previously stated by Hochbaum (2024), our results challenge the common assumption that approximation algorithms are always preferable for large graphs. We demonstrate that exact algorithms can achieve optimal solutions with runtimes only slightly higher than those of approximations like Greedy++, making them a viable alternative when solution quality is critical.

Additionally, we systematically analyze the effectiveness of graph pruning based on the greedy algorithm by Charikar (2000). We show that pruning offers substantial benefits, especially for unweighted graphs with high degree variance, and support this with more extensive experiments. Specifically, we investigate the combination of pruning followed by these fast flow algorithms—a configuration not previously analyzed. We show that with pruning, exact algorithms on unweighted graphs can consistently outperform the unpruned Greedy++ approximation, while the benefit is far less pronounced for weighted graphs.

Our implementation extends the parametric maximum flow framework by Beines et al. (2025) with new, DSP-specific algorithms. These contributions provide both theoretical insights into the performance of exact versus approximate algorithms and practical guidance for practitioners.

2. Preliminaries

2.1. Densest Subgraph Problem

The density of a graph is the ratio between its number of edges and number of nodes (Picard et al. 1982; Goldberg 1984). Given a graph $G = (V, E)$ the goal of the *Densest Subgraph Problems* (DSP) is to find a subgraph $S \subseteq V$ that maximises the ratio between edges and vertices:

$$\max_{S \subseteq V} \frac{E(S)}{|S|} \quad (2.1)$$

Where $E(S)$ is the sum of the edges inside the subgraph S . This problem can be generalized to edge weighted graphs where $E(S)$ then is defined as

$$E(S) = \sum_{(u,v) \in E: u,v \in S} w_{u,v} \quad (2.2)$$

with $w_{u,v}$ the weight of the edge $\{u, v\}$ ($w_{u,v} = 1$ in unweighted graphs).

For later use we define

$$W := E(V) = \sum_{(u,v) \in E} w_{u,v} \quad (2.3)$$

as the (weighted) sum of all Edges. In an unweighted graph $W = |E|$ holds.

The problem is equivalent to finding the subgraph with the maximum average vertex degree. For a subgraph $G' = (V', E')$ the density is exactly half its average degree, since every edge gets included in the degree of two vertices.

$$\frac{\sum_{v \in V'} \deg_{G'}(v)}{|V'|} = \frac{2 \sum_{e \in E'} w(e)}{|V'|} \quad (2.4)$$

Since all exact algorithms for DSP in this thesis are based on a reduction to the parametric maximum flow problem we first need to introduce it.

2.2. Maximum Flow and Minimum Cut

Definition 1 (flow network). A **flow network** $G = (V, E, c, s, t)$ consists of a directed graph $G' = (V, E)$ a source $s \in V$, a sink $t \in V$ and a capacity function $c : E \rightarrow \mathbb{R}_{\geq 0}$.

Definition 2 (maximum flow). Given a flow network a **flow** is a function $f : E \rightarrow \mathbb{R}_{\geq 0}$ satisfying:

1. Capacity constraint: For all $(u, v) \in E$, $0 \leq f(u, v) \leq c(u, v)$.
2. Flow conservation: For all $v \in V \setminus \{s, t\}$,

$$\sum_{u \in V} f(u, v) = \sum_{w \in V} f(v, w)$$

The **value of a flow** is defined as:

$$|f| = \sum_{v \in V} f(s, v) - \sum_{u \in V} f(u, s)$$

A **maximum flow** is a flow f of maximum value $|f|$ from s to t subject to the above constraints.

Definition 3 (residual graph). Given a flow network G and a flow the **residual graph** is a graph constructed by inverting every edge in G which gets the residual capacity $c_{u,v} = c_{v,u} - f(v, u)$.

Definition 4 ((s, t)-Cut). An (s, t) -**cut** is a partition of the vertex set V into two disjoint subsets S and T such that:

$$s \in S, \quad t \in T, \quad S \cup T = V, \quad S \cap T = \emptyset.$$

The set of edges crossing the cut is:

$$C = \{(u, v) \in E \mid u \in S, v \in T\}.$$

The capacity or cost of the cut (S, T) is defined as:

$$c(S, T) = \sum_{(u,v) \in C} c(u, v).$$

A **minimum (s, t)-cut** is a cut that minimizes $c(S, T)$ over all valid cuts, and a **sink-minimal minimum cut** if $|T|$ is minimal among all minimum cuts.

Theorem 1 (Max-Flow Min-Cut Theorem). Let $G = (V, E)$ be a directed graph with a source $s \in V$, a sink $t \in V$, and a capacity function $c : E \rightarrow \mathbb{R}_{\geq 0}$. Then the maximum value of a feasible flow from s to t is equal to the minimum capacity of an (s, t) -cut:

$$\max \{|f| \mid f \text{ is a feasible } (s, t)\text{-flow}\} = \min \{c(S, T) \mid (S, T) \text{ is an } (s, t)\text{-cut}\}$$

Proof. Let f be any feasible flow in G and let (S, T) be any (s, t) -cut. By the flow decomposition and flow conservation, we have:

$$|f| = \sum_{u \in S, v \in T} f(u, v) - \sum_{u \in T, v \in S} f(u, v) \leq \sum_{u \in S, v \in T} c(u, v) = c(S, T)$$

This implies that the value of any flow is upper-bounded by the capacity of any cut:

$$|f| \leq c(S, T)$$

To prove equality, we use the Ford-Fulkerson (Ford Jr et al. 1956) method. After all augmenting paths have been exhausted, let S be the set of vertices reachable from s in the residual graph, and $T = V \setminus S$. Then (S, T) is a cut and no more flow can be sent across it, hence:

$$|f| = c(S, T)$$

This flow is maximum and the cut is minimum. $\square$

2.3. Parametric Flow

This concept can be expanded that the edge capacity is described by a function called flow function. The following expansion of parametric capacities is based on the definition by Beines et al. (2025).

Definition 5 (Monotone Parametric Flow Network). *A **parametric flow network** $G = (V, E, c, [\lambda_{\min}, \lambda_{\max}], s, t)$ differs from a constant flow network in the capacity of each edge e being a function $c(e) : [\lambda_{\min}, \lambda_{\max}] \rightarrow \mathbb{R}_{\geq 0}$. We say that c (and thereby G) is (source-sink) monotone if*

- $c(s, v)$ is non-decreasing for all $v \in V \setminus \{t\}$,
- $c(v, t)$ is non-increasing for all $v \in V \setminus \{s\}$,
- $c(v, u)$ is constant for all $u \in V \setminus \{s\}, v \in V \setminus \{t\}$.

Gallo et al. (1989) observe that if the network is source-sink monotone, the *nesting property* holds:

Observation 1 (Nesting Property). *Let $G = (V, E, c, [\lambda_{\min}, \lambda_{\max}], s, t)$ be a monotone parametric flow network and $\lambda_1, \lambda_2 \in [\lambda_{\min}, \lambda_{\max}]$ two parameter values with $\lambda_1 < \lambda_2$. Further, let (V_s^1, V_t^1) and (V_s^2, V_t^2) be the sink-minimal minimum cuts in $G[\lambda_1]$ and $G[\lambda_2]$, respectively. Then*

$$V_s^1 \subseteq V_s^2 \quad \text{and} \quad V_t^1 \supseteq V_t^2.$$

Definition 6 (Breakpoint Function). *A **breakpoint function** is a function $\beta : V \rightarrow \mathbb{R}_{\geq 0}$ that holds:*

$$C(\beta, \lambda) := (V_s(\beta, \lambda), V_t(\beta, \lambda)) \quad \text{with} \tag{2.5}$$

$$V_s(\beta, \lambda) := \{v \in V \mid \beta(v) \leq \lambda\}, \quad V_t(\beta, \lambda) := \{v \in V \mid \beta(v) > \lambda\}. \tag{2.6}$$

We call a parameter value λ a **breakpoint** if $\beta(v) = \lambda$ for at least one vertex v .

Hochbaum (2008) differentiates between two types of **monotone parametric minimum cut (MPMC)**.

In the *complete parametric analysis* the objective is to calculate the breakpoint function β for the monotone parametric flow network G , without knowing the breakpoints in advance. The *simple sensitivity analysis* takes a series of monotone parameter values and calculates the cuts for each value. For this version algorithms may use the nested property to continue from the last solution with the calculation.

To this point only the *push-relabel* (PR) (Gallo et al. 1989), *pseudoflow* (PF) (Hochbaum 2008) and *incremental breadth-first search* (IBFS) (Goldberg et al. 2011) support this continuation.

For the implementation, another property of flow networks is relevant: if we reverse every edge—swapping each edge’s source and target—and interchange the source and sink nodes, we obtain an equivalent network with the same maximum flow and the same minimum cut. Naturally, the roles of source and sink are also exchanged.

Theorem 2 (Flow Network Reversal Symmetry). *Let $G = (V, E)$ be a directed graph with source $s \in V$, sink $t \in V$, and a capacity function $c : E \rightarrow \mathbb{R}_{\geq 0}$. Construct a new graph $G' = (V, E')$ by reversing every edge in G , i.e., $E' = \{(v, u) \mid (u, v) \in E\}$, and define $c'(v, u) = c(u, v)$. Let the source and sink in G' be $s' = t$ and $t' = s$, respectively. Then:*

$$\max \{|f| \mid f \text{ is a feasible } (s, t)\text{-flow in } G\} = \max \{|f'| \mid f' \text{ is a feasible } (s', t')\text{-flow in } G'\}$$

and the minimum (s, t) -cut in G has the same capacity as the minimum (s', t') -cut in G' .

Proof. Let f be a feasible flow in G . Define a flow f' in G' by $f'(v, u) := f(u, v)$ for each edge $(v, u) \in E'$. Since the edges are reversed and capacities are preserved, f' satisfies:

- **Capacity constraints:** $f'(v, u) = f(u, v) \leq c(u, v) = c'(v, u)$
- **Flow conservation:** For all $v \in V \setminus \{s', t'\}$, net flow is zero because reversing flow preserves conservation

Thus, f' is a feasible (s', t') -flow in G' with value:

$$|f'| = \sum_{v \in V} f'(s', v) = \sum_{v \in V} f(t, v) = - \sum_{v \in V} f(v, t) = |f|$$

Therefore, the maximum flow in G equals that in G' . There cannot be any bigger flow, because then this bigger flow would have also been a bigger flow in G .

Now consider a minimum (s, t) -cut in G given by partition (S, T) with $s \in S, t \in T$. Its capacity is:

$$c(S, T) = \sum_{u \in S, v \in T} c(u, v)$$

In G' , the cut (T, S) (now with $s' = t \in T, t' = s \in S$) has capacity:

$$c'(T, S) = \sum_{v \in T, u \in S} c'(v, u) = \sum_{u \in S, v \in T} c(u, v) = c(S, T)$$

Hence, the minimum cut value is preserved under reversal and the set of cut edges is structurally equivalent.

This proves both the equality of maximum flow values and the correspondence of minimum cut capacities. $\square$

2.4. Solving DSP with Reduction to Parametric Flow

A fundamental approach for exact DSP solving relies on a *reduction to the parametric flow problem*. This method forms the basis for all such solutions that use maximum flow algorithms. It was first described by Goldberg (1984) and our description is based on the one provided by Lanciano et al. (2024) but the difference is not essential. The notation is changed to be consistent with the rest of this thesis.

Given a (weighted) graph $G' = (V, E)$ we construct a parametric flow network $G = (U, A, c, [\lambda_{\min}, \lambda_{\max}], s, t)$:

$$U = V \cup \{s, t\}, \quad A = A_s \cup A_E \cup A_t,$$

with

$$A_s = \{(s, v) \mid v \in V\}, \quad A_E = \{(u, v), (v, u) \mid \{u, v\} \in E\}, \quad A_t = \{(v, t) \mid v \in V\},$$

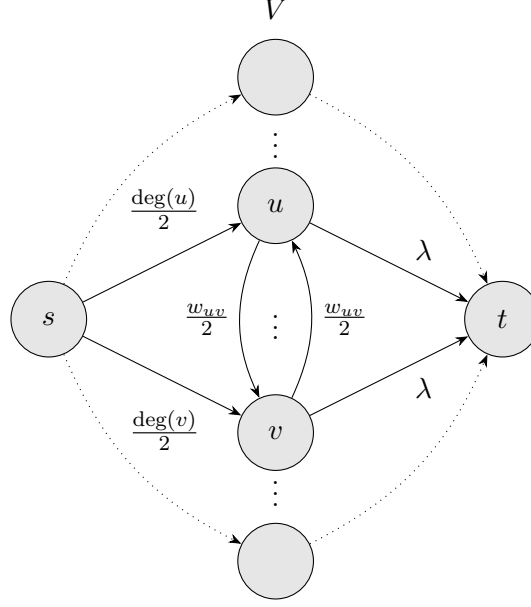
and $c : A \rightarrow \mathbb{R}_+$ defined by

$$c_{ij} := c(e) = \begin{cases} \frac{\deg(v)}{2} & \text{if } e = (s, v) \in A_s, \\ \frac{w_e}{2} & \text{if } e \in A_E, \\ \lambda & \text{if } e \in A_t. \end{cases}$$

For $v \in V$, let $\deg(v)$ denote the (weighted) degree of v in the graph G . The interval $[\lambda_{\min}, \lambda_{\max}]$ here does not really matter for now, only that it includes the optimal value. This graph is illustrated in Figure 2.1.

Let's determine what exactly we look for in this graph.

Theorem 3. *Let (X, Y) be a s - t cut of the flow network G at a parameter value λ , $S = X \setminus \{s\}$ and $T = (V \setminus S) \setminus \{t\}$. Then it holds that $\text{cost}(X, Y) = W + \lambda|S| - E(S)$. In particular, when $X = \{s\}$ then $\text{cost}(X, Y) = W$ holds.*

Figure 2.1.: The parametric flow network constructed from G and λ .

Proof.

$$c(X, Y) = \sum_{i \in X, j \in Y} c_{ij} \quad (2.7a)$$

$$= \sum_{j \in T} c_{sj} + \sum_{i \in S} c_{it} + \sum_{i \in S, j \in T} c_{ij} \quad (2.7b)$$

$$= \sum_{j \in T} \frac{\deg(j)}{2} + |S| \cdot \lambda + \sum_{i \in S, j \in T} \frac{w_{ij}}{2} \quad (2.7c)$$

$$= \sum_{i, j \in T} w_{ij} + \sum_{i \in T, j \in S} \frac{w_{ij}}{2} + \sum_{i \in S, j \in T} \frac{w_{ij}}{2} + |S| \cdot \lambda \quad (2.7d)$$

$$= W - \sum_{i, j \in S} w_{ij} + |S| \lambda \quad (2.7e)$$

$$= W - E(S) + |S| \lambda \quad (2.7f)$$

□

Let (X, Y) be the minimum s - t cut of the flow network for the parameter λ . If $X = \{s\}$ does not hold, then $S = X \setminus \{s\}$ is a vertex subset that satisfies

$$\lambda |S| - E(S) \leq 0 \Leftrightarrow \lambda \leq \frac{E(S)}{|S|}. \quad (2.8)$$

We are therefore looking for the largest parameter value λ so that S is not empty, which corresponds exactly to the largest breakpoint in the flow network.

Since for some algorithms we need to start at a value above this optimum, we need to find an upper bound.

Theorem 4. *Given a (weighted) graph $G = (V, E)$ the density of the densest subgraph is bounded above by $u = \frac{\max\{\deg(v) | v \in V\}}{2}$.*

Proof. Let $G' = (V', E', w')$ be any subgraph of G .

The sum of the weighted degrees of the vertices in G' is $\sum_{v \in V'} \deg_{G'}(v) = 2 \sum_{e \in E'} w_e$.

For any vertex $v \in V'$, its degree in the subgraph cannot be higher than its weighted degree in the original graph, because the edge set of the subgraph is a subset of the original graph's edge set. That is,

$$\deg_{G'}(v) \leq \deg_G(v). \quad (2.9)$$

Furthermore, the weighted degree of any vertex in the original graph is bounded above by the maximum weighted degree of any vertex in G :

$$\deg_G(v) \leq \max_{u \in V} \deg_G(u). \quad (2.10)$$

Combining these inequalities, we have

$$\deg_{G'}(v) \leq \max_{u \in V} \deg_G(u). \quad (2.11)$$

Now, let's consider the average weighted degree of the subgraph G' :

$$\frac{2 \sum_{e \in E'} w(e)}{|V'|} = \frac{\sum_{v \in V'} \deg_{G'}(v)}{|V'|} \quad (2.12a)$$

$$\leq \frac{\sum_{v \in V'} \max_{u \in V} \deg_G(u)}{|V'|} \quad (2.12b)$$

$$= \frac{|V'| \cdot \max_{u \in V} \deg_G(u)}{|V'|} \quad (2.12c)$$

$$= \max_{u \in V} \deg_G(u). \quad (2.12d)$$

Since the density of G' is half of its average weighted degree, we have:

$$d(G') = \frac{\sum_{e \in E'} w(e)}{|V'|} = \frac{1}{2} \cdot \frac{2 \sum_{e \in E'} w(e)}{|V'|} \quad (2.13a)$$

$$\leq \frac{1}{2} \cdot \max_{u \in V} \deg_G(u). \quad (2.13b)$$

This inequality holds for any arbitrary subgraph G' . Therefore, it must also hold for the densest subgraph, as it is also a subgraph of G . Thus, the density of the densest subgraph is bounded above by $u = \frac{\max\{\deg(v) | v \in V\}}{2}$. $\square$

3. Exact Algorithms

3.1. Goldberg's Maximum-Flow-Based Algorithm

Goldberg (1984)'s algorithm does not use a parametric max-flow approach. Instead, it applies a binary search over the possible density values, requiring a logarithmic number of calls to a standard max-flow algorithm to find the solution.

Goldberg also shows that if the difference between the upper and lower bounds is less than $\frac{1}{n(n+1)}$, the lower bound already contains the optimal solution. However, since this detail is not relevant for this thesis, the proof is omitted.

The time complexity of the algorithm is given by $O(T_{\text{Flow}} \log n)$, where T_{Flow} denotes the time complexity of the static max-flow algorithm used in each iteration.

The number of iterations i of the while-loop is the smallest non-negative integer satisfying

$$\frac{1}{2^i} \left(2\lambda_{\text{ub}}^{(0)} - \lambda_{\text{lb}}^{(0)} \right) < \frac{1}{n(n-1)}. \quad (3.1)$$

Therefore, for unweighted graphs,

$$i = O(\log(mn(n-1))) = O(\log n), \quad (3.2)$$

since $m = O(n^2)$.

For weighted graphs, let l denote the length (in bits) of the input. For any reasonable encoding, we assume $l \geq n$. Then the sum of the weights W can be bounded by 2^l , leading to:

$$i = O(\log(2^l n(n-1))) = O(l). \quad (3.3)$$

Clearly, the time complexity of each iteration is dominated by the cost of the max-flow computation, T_{Flow} .

3.2. Parametric Max-Flow for DSP

Goldberg's Algorithm performs $\log n$ calls to a static maximum-flow algorithm. It used binary search to find the specific parameter value λ . This value is achieved when the difference between upper and lower bound (u and l) is less than $\frac{1}{n(n-1)}$. At this point in the cuts (X_u, Y_u) is $X_u = \{S\}$ and (X_l, Y_l) is $X_l \neq \{S\}$. If we look at the constructed graph as a monotone parametric flow network we search for the *breakpoint of highest value*. At this breakpoint the final vertices switch from X to Y and the flow value must be $C(X, Y) = W$.

This property helps us to construct two new algorithms:

3.2.1. Solving the DSP by finding the first breakpoint

Beines et al. (2025) present the algorithm *Parametric Breadth First Search* (PBFS). This algorithm takes ideas from IBFS (Goldberg et al. 2011) and computes the breakpoints in monotone order. This is the first algorithm which achieves this. Other parametric flow algorithms which compute the breakpoints like the Goldberg-Tarjan algorithm (Gallo et al. 1989), Dichotomic Scheme (Eisner et al. 1976) compute the breakpoints in arbitrary order and are therefore unusable for this specific application.

We also consider another new algorithm *Parametric Pseudoflow* (PPF), provided by the supervisor. This algorithm, whose academic paper is not yet published (Sauer 2025), is based on the pseudoflow algorithm (PF) by Hochbaum (2008) and achieves the same result as PBFS.

Both these algorithm find the breakpoints in *ascending* order. The network described in Section 2.4 is not immediately suitable for these algorithms, since the first breakpoint identified would correspond to the smallest value. Thus, we construct $G = (U, A, c, [\lambda_{\min}, \lambda_{\max}], s, t)$ as described in Section 2.4 but with one difference:

Let $u = \frac{\max\{\deg(v) | v \in V\}}{2}$ be an upper-bound of the optimal value

$$c(e) = \begin{cases} \frac{\deg(v)}{2} & \text{if } e = (s, v) \in A_s, \\ \frac{1}{2} & \text{if } e \in A_E, \\ u - \lambda & \text{if } e \in A_t. \end{cases} \quad (3.4)$$

and

$$[\lambda_{\min}, \lambda_{\max}] = [0, u]. \quad (3.5)$$

In this parametric flow network the breakpoint with smallest value λ found is the biggest breakpoint in the original construction. The optimal density λ^* is then calculated as:

$$\lambda^* = u - \lambda \quad (3.6)$$

It is easy to modify PBFS and PPF in a way that it terminates after finding the first breakpoint. We call these versions *first-breakpoint PBFS* (FPBFS) and *first-breakpoint PPF* (FPPF) and bundle all algorithms which finds the first breakpoint as ***first-breakpoint flow algorithms (FBA)***. The pseudocode of the resulting algorithm we call *DSP with First-Breakpoint Algorithm* (DSP-FBA) is described in Algorithm 1.

The overall asymptotic complexity of this algorithm is $\mathcal{O}(\text{complexity}(FBA))$. It is determined by two main phases:

1. **Flow Network Construction:** The creation of the flow network has a time complexity of $\mathcal{O}(2m + 2n)$, which simplifies to $\mathcal{O}(m + n)$.
2. **FBA Algorithm:**
 - The complexity of the FPBFS algorithm is $\mathcal{O}(n^2m)$, the same as PBFS (Beines et al. 2025).

Algorithm 1: DSP-FBA

Input: Graph $G = (V, E)$
Input: Densest Subgraph $S \subseteq V$ and its density λ^*

- 1 Construct the parametric flow network as described
- 2 Run FBA to find the first breakpoint
- 3 $\lambda^* \leftarrow u - \text{FirstBreakpoint}$
- 4 $S \leftarrow \emptyset$
- 5 **for** each $v \in V$ **do**
- 6 **if** $\text{breakpoint}(v) = \text{FirstBreakpoint}$ **then**
- 7 $S \leftarrow S \cup \{v\}$
- 8 **return** S, λ^*

- The complexity of the FPPF algorithm is $\mathcal{O}(n^3m)$, the same as PPF (Sauer 2025). Actually they propose a version with complexity $\mathcal{O}(n^2m)$, but in practise that version was slower.

Since the runtime of the second phase dominates the first phase, the total complexity is that of the first-breakpoint algorithm used.

3.2.2. DSP with an Intersection Scheme and Restartable Maxflow Algorithm

A different approach is to use the nesting property and an iterative refinement scheme to find the largest breakpoint in the flow function. Consider the graph constructed in the Charikar algorithm. If we plot the maximum flow value on the y-axis with respect to the parameter λ on the x-axis, the resulting function is piecewise linear and monotonically increasing. The slope changes at breakpoints corresponding to structural changes in the minimum cut. If the source compartment of the cut includes only the source, then all outgoing edges are in the cut, therefore the flow value must be W .

We begin at the origin with $\lambda = 0$ and examine the flow function of the cut edges. Adding all their flow functions together we get the *flow function of the cut*. By intersecting this function with a constant line at $f(\lambda) = W$, we iteratively refine our estimate for λ . In each step, we compute the intersection point, update λ accordingly, and run the restartable flow algorithm.

With this approach the sink incident edges would gain capacity as λ grows. This contradicts the definition of our monotone parametric flow network and is also not supported by the implementations used. Thus, we construct the equivalent problem by swapping the direction of every edge and swap sink and source, like described in Theorem 2. Once the maximum flow equals W , in the sink component is either the densest subgraph and the sink or just the sink left, because floating-point rounding can place the computed value just before or just after the breakpoint, potentially leaving only the sink. The sink side of the last cut before the final minimum cut defines the densest subgraph excluding

the sink, and the last λ value its density. The first step of this algorithm is drawn in Figure 3.1

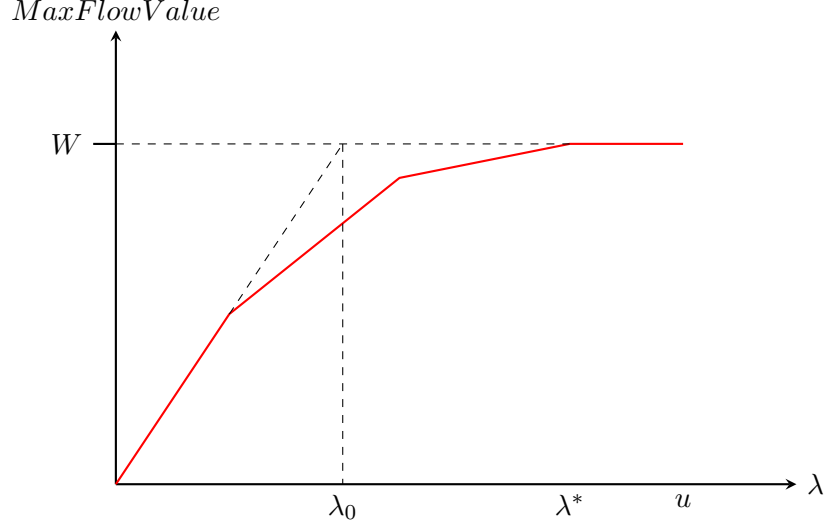


Figure 3.1.: First Iteration of DSP-Intersect (Algorithm 2) to find the biggest breakpoint.

Algorithm 2: DSP-Intersect

Input: Graph $G = (V, E)$

Output: Densest subgraph $S \subseteq V$, density λ^*

```

1 Construct parametric flow network as described
2  $\lambda \leftarrow 0$ 
3 repeat
4   UPDATENETWORK( $\lambda$ )
5   Run RESTARTABLEMAXFLOW on the network
6   Let  $f(\lambda)$  be the flow function of the resulting cut
7    $\lambda \leftarrow \text{COMPUTEINTERSECTIONPOINT}(f(\lambda), W)$ 
8 until  $\text{MaxFlow}(\lambda) = W$ 
9 Let  $(X, Y)$  be the second to last minimum cut and  $S = Y \setminus \{t\}$ 
10  $\lambda^* \leftarrow \lambda$ 
11 return  $S, \lambda^*$ 
```

For the resulting algorithm described in Algorithm 2 any restartable max flow algorithm supporting this continuation can be used, in our implementation we compared the push-relabel (Goldberg et al. 1988) implementation from Beines et al. (2025) and the pseudoflow (Hochbaum 2008) implementation from Sauer (2025). Other algorithms with this property could be used, like IBFS (Goldberg et al. 2011).

To enhance efficiency, the algorithm can be initialized with a better lower bound for the optimal density. The calculation can then start at this higher value rather than at $\lambda = 0$. The density of the entire graph, for instance, serves as a practical choice for this

initial parameter.

3.2.3. Incremental Parametric Cut (IPC)

Hochbaum (2024) presents the *Incremental Parametric Cut* procedure (IPC) for multiple different ratio problems. They also show an alternative construction of a parametric flow network with just $2 \cdot n + m$ arcs instead of the $2 \cdot (n + m)$ arcs as in the version presented above. We omit this change because in the implementation of the graph structure used in this paper provided by Beines et al. (2025) for every edge a reverse edge must be inserted if not yet there and it does not directly support the more complex flow functions needed for the smaller construction. Therefore, we use the version as described above for our experiments.

The algorithm fitted for DSP is shown in Algorithm 3.

Algorithm 3: Incremental Parametric Cut (IPC)

Input: (weighted) Graph $G = (V, E)$

Output: Densest subgraph $S \subseteq V$, density λ^*

```

1  $\lambda \leftarrow 0$ 
2 Construct parametric flow network  $(U, A, c, [0, u])$ 
3 repeat
4   UPDATENETWORK( $\lambda$ )
5   Run RESTARTABLEMAXFLOW on the network
6   Let  $(X, Y)$  be the minimum cut and  $S = T \setminus \{s\}$ 
7    $\lambda \leftarrow \frac{E(S)}{|S|}$ 
8 until  $E(S) - \lambda \cdot |S| = 0$ 
9 Let  $(X, Y)$  be the second to last minimum cut and  $S = Y \setminus \{t\}$ 
10  $\lambda^* \leftarrow \lambda$ 
11 return  $S, \lambda^*$ 
```

Algorithm 3 (IPC) (Hochbaum 2024) and Algorithm 2 (DSP-Intersect) are almost the same in design, differing only in the underlying calculations while yielding identical parameter sequences.

Theorem 5. *The Algorithm 2 and the Algorithm 3 produce the same sequence of parameter values.*

Proof. At each iteration they both run a restartable maximum flow algorithm. Let X, Y be the resulting min-cut. Let $\text{cost}(X, Y)$ denote the cost of the cut. DSP-Intersect then solves this equation:

$$\text{cost}(X, Y) = W \tag{3.7}$$

With Theorem 3 that rewrites to:

$$W + \lambda|S| - E(S) = W \Leftrightarrow \lambda = \frac{E(S)}{|S|} \tag{3.8}$$

This is exactly the calculation Algorithm 3 performs. □

3.2.4. Difference IPC vs DSP-Intersect and Runtimes

The primary difference between the IPC and DSP-Intersect algorithms for finding the densest subgraph lies in how they calculate the next parameter value λ . IPC needs to sum the edges within the sink component, the one containing the densest subgraph, while DSP-Intersect needs to calculate the whole flow value of the cut as well as its function. Both algorithms must track the sink components, because there is no guarantee that the sink at termination still contains the densest subgraph. Due to floating-point operations, the actual value might fall just before or just after the breakpoint, potentially leaving only the sink. Thanks to the nesting property, the complexity of both IPC and DSP-Intersect matches that of a single minimum-cut procedure $T(n, m)$. More precisely, the overall complexity is

$$T(n, m) + \mathcal{O}(q(n + m)), \tag{3.9}$$

where q is the number of parameter values visited. The additional $n + m$ term arises because computing each calculation of the next parameter value takes $\mathcal{O}(m)$ time and updating the flow network takes $\mathcal{O}(n)$. Building the flow network is asymptotically negligible compared with these terms. The implemented flow algorithms are:

- The **Push-relabel** implementation runs in $\mathcal{O}(n^2m)$ (Goldberg et al. 1988) and is taken from Beines et al. (2025).
- The specific pseudoflow implementation by Beines et al. (2025) discussed has a worst-case runtime of $\mathcal{O}(n^3m)$. This variant deviates from a version with a better theoretical bound of $\mathcal{O}(n^2m)$ because it is much faster in typical, practical scenarios (Sauer 2025).

4. Approximation Algorithms

The densest subgraph problem is polynomial-time solvable, yet many approximation algorithms exist as exact approaches were often viewed as inefficient.

4.1. Charikars Greedy Peeling Algorithm

Charikar (2000) proposes a simple greedy algorithm for the densest subgraph problem: The algorithm starts with the full graph and iteratively removes the node with the smallest degree. At each step, it records the density of the remaining subgraph. After all nodes have been removed, it returns the subgraph with the highest density encountered during the process. The pseudocode is given in Algorithm 4.

Algorithm 4: Greedy Peeling Algorithm

Input: $G = (V, E)$

Output: An approximately densest subgraph of G : G_{densest} and its density ρ^*

```

1  $S_n \leftarrow V$ 
2  $i \leftarrow n$ 
3 while  $i > 1$  do
4    $v_{\min} \in \arg \min \{\deg_{S_i}(v) \mid v \in S_i\}$ 
5    $S_{i-1} \leftarrow S_i \setminus \{v_{\min}\}$ 
6    $i \leftarrow i - 1$ 
7  $S_{\max} \in \arg \max \{\text{density}(S) \mid S \in \{S_1, \dots, S_n\}\}$ 
8  $\rho^* \leftarrow \rho(S_{\max})$ 
9 return  $S_{\max}, \rho^*$ 

```

Theorem 6. *Algorithm 2 is a 2-approximation and for unweighted graphs can be implemented in linear time.*

The proof of Theorem 6 largely follows the approach presented by Lanciano et al. (2024), with adjustments to notation.

Proof. Let $S^* \subseteq V$ be an optimal solution to DSP. From the optimality of S^* , for any $v \in S^*$, we have for the density ρ

$$\rho(S^*) = \frac{E[S^*]}{|S^*|} \geq \frac{E[S^* \setminus \{v\}]}{|S^*| - 1} = \rho(S^* \setminus \{v\}). \quad (4.1)$$

Using $E[S^* \setminus \{v\}] = E[S^*] - \deg_{S^*}(v)$, where $\deg_S(v)$ denotes the degree of v in the subgraph $G[S \subseteq V]$. We can transform this equality such that for any $v \in S^*$

$$\deg_{S^*}(v) \geq \rho(S^*) \cdot |S^*|. \quad (4.2)$$

Let v^* be the vertex that is contained in S^* and removed first by Algorithm 4. Let $S' \subseteq V$ be the vertex subset kept just before removing v^* in Algorithm 4. Then the density of S' can be evaluated as

$$\rho(S') = \frac{1}{2} \frac{\sum_{v \in S'} \deg_{S'}(v)}{|S'|} \geq \frac{1}{2} \frac{|S'| \deg_{S'}(v^*)}{|S'|} \geq \frac{1}{2} \deg_{S^*}(v^*) \geq \frac{1}{2} \rho(S^*), \quad (4.3)$$

where the first inequality follows from the greedy choice of $v^* \in S'$, the second inequality follows from $S' \supseteq S^*$, and the last inequality follows from inequality 4.2. Since S' is one of the candidate subsets of the output, this shows a 2-approximation.

For unweighted graphs, the algorithm can be implemented to run in linear time.

For each possible degree $d = 0, \dots, n-1$, the vertices with degree d are stored in a doubly linked list. The algorithm scans these lists in increasing order of d until it finds a vertex. Once a vertex v is found and removed, each of its neighbors is moved to the one lower list. This operation takes $\mathcal{O}(\deg(v))$ time.

In the next iteration, the algorithm only needs to start scanning from the one list lower than the current degree list, since no vertex can exist in a list corresponding to a lower degree than that. Thus, the total number of moves between lists over all operations is bounded by $\mathcal{O}(n)$.

Therefore, the total runtime of the Algorithm is $\mathcal{O}(m+n)$.

The graph density $\rho(V)$ does not need to be recomputed from scratch in each iteration. It is calculated once at the beginning and then updated incrementally: In each step one node is removed, decreasing the denominator by 1, and the sum of degrees in the numerator is adjusted by subtracting the degree of the removed node. $\square$

Theorem 7. *The approximation factor 2 of Algorithm 4 is tight.*

Proof. Boob et al. (2020) provide an instance that shows that this approximation is tight.

Consider the Graph $G = B \cup (\bigcup_{i=1}^k H_i)$ with $B = K_{d,D}$ is a bipartite complete $d \times D$ graph and k $d+2$ -cliques $H_i = K_{d+2}$.

When $d \ll D$ and $k \rightarrow \infty$ the density of G is

$$\frac{2dD + (d+1)(d+2)k}{2d + 2D + 2k(d+2)} \rightarrow \frac{d+1}{2}.$$

This is the density of any $d+2$ -clique. Charikars Algorithm outputs G itself since it starts removing nodes from B and therefore never sees a subgraph with a higher density. However, the density of B is $\frac{dD}{d+D} \approx d$, which is the optimal solution. Therefore, the approximation is tight. $\square$

4.2. Iterative Peeling Algorithm Greedy++

Boob et al. (2020) proposed the Algorithm *Greedy++* for DSP based on Charikars greedy peeling algorithm and inspired by Arora et al. (2012) multiplicative weights update.

The algorithm iteratively applies a modified peeling procedure for a total of T passes. The first pass is identical to Charikar's standard greedy algorithm, where vertices are

successively removed in increasing order of their degree. The key innovation of Greedy++ lies in subsequent passes. It maintains a *load* for each vertex, which is updated at the end of each pass. In any given pass i , the algorithm peels the vertex u that minimizes the sum of its current degree in the remaining graph, $\deg_H(u)$, and its accumulated load from the previous pass, $\ell_u^{(i-1)}$. This new minimum value is then stored as the vertex's load for the next pass, $\ell_u^{(i)}$. By incorporating this memory of past peeling orders, Greedy++ avoids repeatedly making the same greedy choices, allowing it to explore different removal sequences and discover denser subgraphs. The pseudocode of Greedy++ is given in Algorithm 5.

Algorithm 5: Greedy++

Input: Undirected graph G , iteration count T

Output: An approximately densest subgraph of G : G_{densest} and its density ρ^*

```

1  $G_{\text{densest}} \leftarrow G$ 
2 Initialize the vertex load vector  $\ell^{(0)} \leftarrow 0 \in \mathbb{Z}^n$ 
3 for  $i \leftarrow 1$  to  $T$  do
4    $H \leftarrow G$ 
5   while  $H \neq \emptyset$  do
6     Find the vertex  $u \in H$  with minimum  $\ell_u^{(i-1)} + \deg_H(u)$ 
7      $\ell_u^{(i)} \leftarrow \ell_u^{(i-1)} + \deg_H(u)$ 
8     Remove  $u$  and all its adjacent edges  $\{u, v\}$  from  $H$ 
9     if  $\rho(H) > \rho(G_{\text{densest}})$  then
10        $G_{\text{densest}} \leftarrow H$ 
11        $\rho^* \leftarrow \rho(H)$ 
12 return  $G_{\text{densest}}, \rho^*$ 

```

The runtime of the *Greedy++* algorithm is a major advantage. It is implemented to run efficiently with each iteration taking $\mathcal{O}((n + m) \cdot \min(\log n, T))$ time.

The deletion of a vertex and the subsequent update of its neighbors' degrees takes $\mathcal{O}(m)$ time across an entire iteration, as each edge is processed only once. The most crucial part of the algorithm's efficiency is how it finds the vertex with the minimum degree in each step. There are two primary approaches for this:

1. **Using Degree Lists:** For unweighted graphs, one can use a set of lists, one for each possible degree value. The algorithm finds the minimum-degree vertex by scanning these lists. After removing a vertex of degree d , it only needs to resume its scan from degree $d - 1$, not from the beginning. This leads to a total time of $\mathcal{O}(mT)$.
2. **Using a Priority Queue:** A more general method for finding the minimum-degree vertex is to use a priority queue or a Balanced Binary Search Tree (BBST). This data structure allows for finding the minimum element (the minimum-degree vertex) and updating its neighbors' degrees in $\mathcal{O}(\log n)$ time per operation. Since there are at most $\mathcal{O}(m)$ edge updates over the course of the algorithm's execution,

the total time complexity for this approach is $\mathcal{O}(m \log n)$. This method is needed for **weighted graphs**, where the concept of simple integer degrees does not apply, making the degree-list approach unfeasible.

Boob et al. (2020) note that the weighted graph implementation, which relies on the BBST, is **6 to 10 times slower** than the unweighted version that uses degree lists. This difference highlights the trade-off between the versatility of a BBST, which works for both unweighted and weighted cases, and the superior speed of a specialized data structure like degree lists for the simpler unweighted case.

Their results demonstrate that this Algorithm performs exceptionally well, consistently achieving 90% of the optimal value within just three iterations. When compared to Goldberg’s Algorithm 3.1 (Goldberg 1984), *Greedy++* is shown to be magnitudes faster.

Chekuri et al. (2022) show that *Greedy++* outputs a $(1 - \epsilon)$ -approximation after $\mathcal{O}\left(\frac{\Delta}{\lambda^* \epsilon^2} \log n\right)$ iterations, where Δ is the maximum degree and λ^* the optimal value in the input graph. Their argument relies on the Lovász extension of a supermodular function and a sequence of technical lemmas that would add considerable length without providing new insights beyond the original presentation. For completeness, we simply state their result and refer the reader to their paper for the full derivation.

5. Graph Pruning

The problem of finding the **Densest Subgraph (DSP)** is computationally challenging, especially when confronted with the **massive scale** of real-world datasets like social or web graphs. To bridge the gap between theoretical guarantees and practical feasibility on massive graphs, **Graph Pruning** acts as an additional preprocessing step. Its primary goal is to drastically reduce the graph size while providing a **guarantee that the true densest subgraph remains intact** within the smaller, pruned structure.

5.1. Pruning Mechanisms: Leveraging Graph Structure

The necessity of pruning was expertly demonstrated by Wu et al. (2017), who focused on making the DSP solvable for massive datasets that exceed single-machine storage capacity. They employed two complementary strategies to reduce the problem size:

5.1.1. Structural Pruning via Graph Partitioning

The first approach exploits the typical properties of real-world graphs (e.g., high locality and clustering). Wu et al. (2017) partition the graph into small, **overlapping sub-graphs** that are *likely* to contain the densest subgraph. This decomposition allows the computationally expensive exact algorithms to be run independently on much smaller, tractable partitions, effectively parallelizing the overall problem using a **MapReduce** framework.

5.1.2. Algorithmic Pruning via Degree Peeling

The second, and theoretically more powerful, approach involves an iterative removal process based on vertex degrees. This mechanism closely resembles the initial steps of Charikar’s greedy approximation algorithm (Charikar 2000).

The process iteratively removes all vertices whose degree is smaller than the **current maximum density** identified so far. The key theoretical breakthrough is that this process can stop once the density no longer increases—a point that marks a significant size reduction—while maintaining the crucial guarantee: **the pruned graph still contains the true densest subgraph**. This provides a fast, degree-based filter that mathematically ensures the preservation of the optimal solution.

5.2. A Pruning-Accelerated Framework for DSP

While Wu et al. (2017) relied on pruning to enable the use of the computationally expensive exact Goldberg Algorithm (Goldberg 1984) on a parallel system, the efficiency gain inspires a compelling, modern two-stage framework for single-machine, high-performance computing:

Stage 1: Fast Pruning and Bounding We propose using a slightly modified version of the Charikar greedy peeling algorithm (Algorithm 4) specifically for preprocessing:

- **Version P-1 (Early-Stop Pruning):** A minimum-effort version that stops as soon as a vertex cannot be removed based on the maximum density found.
- **Version P-2 (Full-Run Pruning and Bounding):** Completes the full run, ensuring that all unnecessary vertices are removed. Additionally, this full run automatically provides a guaranteed **2-approximation** of the optimal density. This result can then be used as the **upper and lower bounds** to accelerate subsequent, more complex DSP algorithms (like DSP-FBA, DSP-Intersect and IPC).

Stage 2: Exact Computation on the Pruned Graph After the rapid preprocessing, a more computationally intensive DSP algorithm such as DSP-Intersect, DSP-FBA, IPC or even Greedy++ itself is executed on the resulting, hopefully **dramatically smaller**, pruned graph.

The central hypothesis for this work is that if the pruning step (Stage 1) is fast and reduces the graph size by a significant margin, the **total runtime** of the two-stage pipeline will be far superior to running the exact algorithm directly on the original graph, while also reducing the overall storage load.

6. Experimental Setup

6.1. Hardware

The experiments were conducted on a server equipped with an **AMD EPYC 7402P 24-Core processor** clocked at **2.80 GHz**. The processor features **24 cores**, each with **2 threads**, and is supported by **256 GiB of RAM**. Cache specifications include **768 KiB L1d**, **768 KiB L1i**, **12 MiB L2**, and **128 MiB L3**. The operating system used was **Ubuntu 22.04.4 LTS**.

The project was implemented in **C++20** and compiled using **CMake version 3.25.1** with **GNU 11.4.0**.

Simexpal 1.1 (Angriman et al. 2019) was used to conduct the experiments.

6.2. Implementation

As basis for the implementation serves the framework for parametric maximum flow created by Beines et al. (2025) in which we included the implementation of pseudoflow and PPF by Sauer (2025). Using these implementation of push relabel, pseudoflow, PBFS and PPF we implemented the Algorithms for DSP as described above. For Greedy++ we based it on the implementation by Boob et al. (2020). For a fairer comparison we adapted this implementation to use the same C++ datastructures and encapsulate it in a class, as we did with the flow based algorithms. Since constructing the flow network (Section 2.4) plays an important role in the runtime for parametric maximum flow, we therefore must also include the initiation of the necessary data structures for Greedy++.

6.3. Instances

For comparability, we use the same instances as Boob et al. (2020). A variety of instances are obtained from the SNAP database (Leskovec et al. 2014), ASU’s Social Computing Data Repository (Zafarani et al. 2009), and the Koblenz Network Collection (Kunegis 2013). Directionality is ignored and duplicate edges are removed. All graphs are treated as unweighted. Boob et al. (2020) also include signed networks but do not explain how they handle these, even though the problem is NP-hard on signed graphs. Thus, in our experiments we ignore edge signs in signed networks. They use one instance from BioGRID (Stark et al. 2006), but do not specify which one, so this instance cannot be used.

All unweighted instances used in our experiments are listed in Table 6.1

instance	n	m	m/n	λ^*	$ S^* $	$var(deg(v))$	u
facebook	4 039	88 234	21.85	77.35	202	2 747.24	523.50
BlogCatalog3 [2]	10 313	333 983	32.38	98.28	910	31 576.36	1 997.00
email-Enron	36 692	183 831	5.01	37.34	555	1 303.21	692.50
soc-Epinions1	75 888	405 740	5.35	60.25	999	1 852.10	1 523.00
soc-sign-Slashdot081106	77 357	468 554	6.06	42.13	205	1 631.77	1 269.50
soc-Slashdot0811	77 360	469 180	6.06	42.27	207	1 636.36	1 270.50
Ego-twitter	81 306	1 342 296	16.51	69.62	545	4 602.74	1 692.50
soc-sign-Slashdot090216	81 871	497 672	6.08	42.78	210	1 643.20	1 274.00
soc-sign-Slashdot090221	82 144	500 481	6.09	42.80	210	1 656.21	1 275.00
soc-Slashdot0902	82 168	504 230	6.14	43.44	219	1 686.77	1 277.00
ego-gplus	107 614	12 238 285	113.72	629.92	4 423	338 147.58	10 064.50
web-Stanford	281 904	1 992 636	7.07	59.39	597	27 796.03	19 313.50
actor-collaboration [1]	382 220	15 038 083	39.34	309.30	2 381	26 675.42	1 979.00
com-dblp	425 957	1 049 866	2.46	56.57	115	100.15	172.50
com-amazon	548 552	925 872	1.69	4.80	97	33.20	275.50
web-BerkStan	685 231	6 649 470	9.70	103.41	392	81 317.47	42 116.00
web-Google	916 428	4 322 051	4.72	28.04	123	1 573.75	3 167.00
roadNet-PA	1 090 920	1 541 898	1.41	1.88	452	1.03	5.50
com-youtube	1 157 828	2 987 624	2.58	45.60	1 959	2 576.00	14 378.00
roadNet-TX	1 393 383	1 921 660	1.38	2.08	26	1.01	7.00
wiki-topcats	1 791 489	25 444 207	14.20	71.93	947	82 543.42	119 172.00
roadNet-CA	1 971 281	2 766 607	1.40	1.97	62	0.99	7.00
wiki-Talk	2 394 385	4 659 565	1.95	114.14	1 384	10 507.96	50 015.50
orkut-links [1]	3 072 442	117 184 899	38.14	227.87	25 969	23 957.19	16 657.50
cit-Patents	6 009 555	16 518 947	2.75	40.13	116	110.06	397.50
livejournal-groupmemberships [1]	7 489 074	112 305 407	15.00	112.88	121 914	722 883.13	526 861.00
trackers [1]	27 665 731	140 613 747	5.08	334.48	1 857	7 694 788.74	5 785 977.50

Table 6.1.: Overview of unweighted instances: n is the number of vertices, m the number of edges, λ^* and $|S^*|$ denote the density and size of the densest subgraph, $var(deg(v))$ the variance of the vertex degree and u is the upper-bound used by DSP-FBA which is half of the maximum node degree. Optimal values are those from the *IPC_{PR}* run. Sources: [1] KONECT (Kunegis 2013), [2] Zafrani (Zafarani et al. 2009); unmarked instances are from SNAP (Leskovec et al. 2014).

The instances **Ego-twitter** and **ego-gplus** contain seemingly arbitrary indices that have up to 12 digits. Because our algorithms cannot process these large numbers, we first converted the indices to a continuous range beginning with zero.

Boob et al. (2020) mention using a Twitter graph set (Sotiropoulos et al. 2019) for weighted graphs, and we include these instances while using the frequency of an edge as its weight.

Since these four weighted instances seem to be a little few, we used all matrices from the Sparse Matrix Collection (SPMC) by Davis et al. (2011) marked as *undirected weighted graph*. All these weighted instances used in our experiments are listed in Table 6.2

instance	n	m	m/n	W/n	λ^*	$ S^* $	$n/ S^* $	$var(deg(v))$	u
GD97_b	46	132	2.87	218.61	1 221.94	6	7.67	1 558 183.29	2 727.67
lesmis	77	254	3.30	5.32	27.18	11	7.00	767.25	80.00
Sandi_authors	86	124	1.44	1.05	4.80	5	17.20	18.57	14.00
Journals	124	5 972	48.16	2 636.53	20 077.75	12	10.33	397 723 730.56	50 459.00
USAir97	332	2 126	6.40	0.23	1.81	45	7.38	2.94	6.67
netscience	1 589	2 742	1.73	0.37	3.54	10	158.90	3.58	16.00
geom	7 343	11 898	1.62	1.54	71.67	6	1 223.83	521.42	349.50
hep-th	8 361	15 751	1.88	0.92	17.01	2	4 180.50	28.09	40.00
Reuters911	13 332	148 038	11.10	12.70	588.12	8	1 666.50	68 051.04	7 501.00
human_gene2	14 340	9 027 024	629.50	13.17	58.24	3 328	4.31	3 871.05	185.37
astro-ph	16 706	121 251	7.26	1.02	15.10	18	928.11	39.22	55.50
cond-mat	16 726	47 594	2.85	0.81	11.17	2	8 363.00	20.90	48.50
human_gene1	22 283	12 323 680	553.05	11.34	62.68	3 775	5.90	3 853.31	218.17
cond-mat-2003	31 163	120 029	3.85	1.01	17.60	2	15 581.50	44.83	73.50
cond-mat-2005	40 421	175 693	4.35	1.11	23.00	2	20 210.50	60.71	96.00
mouse_gene	45 101	14 461 095	320.64	5.41	28.47	4 101	11.00	952.62	135.29
twitter-quotes	16 627 579	657 975	0.04	0.02	180.30	244	68 145.82	1 804.17	2 563.00
twitter-replies	16 636 365	1 002 496	0.06	0.05	292.74	35	475 324.71	1 561.81	2 297.00
twitter-retweets	16 647 272	3 342 154	0.20	0.20	1 654.95	357	46 631.01	39 412.09	11 865.50
twitter-follows	16 853 360	28 395 679	1.68	0.98	443.42	1 657	10 171.01	6 494 338.82	4 974 369.50

Table 6.2.: Overview of weighted instances: n is the number of vertices, m the number of edges, λ^* and $|S^*|$ denote the density and size of the densest subgraph, $var(deg(v))$ the variance of the weighted vertex degree and u is the upper-bound used by DSP-FBA which is half of the maximum node degree. Optimal values are those from the IPC_{PR} run. Sources: all four twitter instances from Sotiropoulos et al. (2019), all other instances from the SPMC (Davis et al. 2011).

6.4. Algorithm Performance Metrics

We conducted a detailed performance analysis of each exact algorithm on every graph instance. For the exact methods, we recorded two primary metrics:

1. the time required to construct the flow network,
2. the runtime of the algorithm.

For *Greedy++*, we evaluated its performance as an approximation algorithm. Given the optimal solution value, we specified target percentages (0.80, 0.90, 0.95, 0.99, 1.00) and measured both runtime and number of iterations until the algorithm achieved at least each target fraction of the optimum. Following Boob et al. (2020) and confirmed by our experiments that three iterations always reach at least 90% of the optimum for the unweighted instances, we also executed *Greedy++* with exactly three iterations and recorded its runtime.

All experiments were repeated on both the original and the *pruned* graphs (see Chapter 5). For pruning we tracked the pruning time and the resulting graph size.

Each algorithm–instance pair was run five times with a 30-minute time limit; we report the median of these runs. For pruning statistics we used the median of all instances and runs.

Throughout this thesis we use the following shorthand:

- **PF**: IPC or DSP-Intersect using pseudoflow, depending on context.
- **PR**: IPC or DSP-Intersect using push-relabel, depending on context.
- **FPPF**: DSP-FBA using parametric pseudoflow.
- **FPBFS**: DSP-FBA using parametric breadth-first search.
- **G-XX**: Greedy++ run until XX% of the optimum is reached (e.g., G-90).
- **G-3**: Greedy++ run for exactly three iterations.
- **P-X**: Pruning Version.

7. Results

We observed that although exact algorithms are designed to produce identical solutions, floating-point operations introduced slight deviations. In nearly all instances, the resulting solution values differed only by a small fraction of one per mille. While the identified densest subgraph was typically the same, exceptions occurred for the largest instances *livejournal-groupmemberships* and *orkut-links*. On these, FPBFS and G-100 did not produce results within the given timeframe, and FPPF’s densest subgraph differed from those found by IPC and DSP-FBA, yielding densities up to 0.003% lower. Additionally, FPBFS was not able to produce a result on the **twitter-follows** instance in the given timeframe. For the absolute runtimes of the compared algorithms see Appendix A.

7.1. Comparing First Breakpoint Algorithms

We run Algorithm 1 (DSP-FBA) with FPBFS based on PBFS by Beines et al. (2025) as well as with FPPF based on PPF by Sauer (2025). Using FPBFS as the baseline we calculated the speedup achieved by FPPF once comparing the whole runtime including the construction of the flow network and once just the runtime of the flow algorithm. Table 7.1 shows the descriptive statistics of these values. We did not observe a remarkable difference between the relative performance on weighted or unweighted instances.

On the **roadNet-PA** instance, FPPF also produced a slightly different solution than IPC and DSP-Intersect: it identified a densest subgraph with 451 vertices instead of 452, resulting in a density lower by 0.014%. This marginal discrepancy is most likely due to floating-point precision issues and is expected to be resolved before the official release of the PPF paper (Sauer 2025). We still consider the solution valid, as the densest subgraph found by *Greedy++* often differs significantly from the optimal one while achieving a comparable or lower density.

	min	10%	25%	mean	median	75%	90%	max	var
Included	1.25	1.40	1.47	1.69	1.54	1.68	1.87	3.50	0.26
Excluded	1.47	1.80	1.97	2.52	2.14	2.92	3.65	5.18	0.77

Table 7.1.: Speedup of DSP-FBA using FPPF in comparison to FPBFS. The value was calculated as follows: $runtime(FPBFS)/runtime(FPPF)$. The difference between the rows is that the first row includes the construction of the flow network, while the second does not. A value greater 1 means FPPF is faster than FPBFS.

Our results clearly show that even though the theoretical runtime of the implemented FPPF is worse than that of FPBFS, the practical performance is better. In practice, FPPF is 2.14 times faster than FPBFS in median, with a speedup ranging from 1.47 to 5.18 times. When we factor in the time to create the flow network (which is the same for both), it is still 1.54 times faster in median, with a range of 1.25 to 3.50 times.

7.2. Comparing IPC and DSP-Intersect with pseudoflow and push-relabel

We run DSP-Intersect as well as IPC each while using PF and PR. We compared the performance of DSP-Intersect and IPC, each using either PF or PR. Focusing solely on the run times (and excluding the flow network construction, which should be the same across all instances but may vary slightly between experiment runs), we observed that IPC was generally faster than DSP-Intersect while using the same flow algorithm (see Table 7.2). This speedup for PF is in median 1.24, while reaching from 0.99 to 1.59. For PR this speedup is 1.31 in median while reaching from 0.99 to 1.71.

	algorithm	min	10%	25%	mean	median	75%	90%	max	var
Excluded	PR	0.99	1.10	1.17	1.31	1.31	1.42	1.56	1.71	0.03
Excluded	PF	0.99	1.10	1.17	1.25	1.24	1.33	1.39	1.59	0.01
Included	PR	0.99	1.06	1.09	1.16	1.17	1.20	1.27	1.33	0.01
Included	PF	0.99	1.06	1.11	1.15	1.15	1.19	1.25	1.29	0.00

Table 7.2.: Speedup of IPC in comparison to DSP-Intersect while excluding and including the creation of the flow network, both for PR and PF. The value was calculated as $runtime(DSP - Intersect)/runtime(IPC)$. A value greater 1 means IPC is faster.

Only on the `human_gene1` instance DSP-Intersect was about 0.6% faster than IPC, therefore nearly identical. We can see in Figure 7.1 a relation between the speedup and the ratio of vertices in the whole graph and inside the densest subgraph. These results indicate that, in IPC, the computation of the next λ value accelerates as the sink compartment becomes smaller, as that is the sole substantial difference between the two algorithms. Therefore, we are going forward only with IPC and excluding DSP-Intersect from the comparisons.

7.3. Comparing IPC in its Versions

The IPC algorithm using **push-relabel (PR)** was generally faster than the version using **pseudoflow (PF)**, as seen in Table 7.3.

For unweighted instances the speedup is in median 1.13, while ranging from 0.71 to 1.42. For weighted it is 1.32, ranging from 0.77 to 1.64. Including the time for the

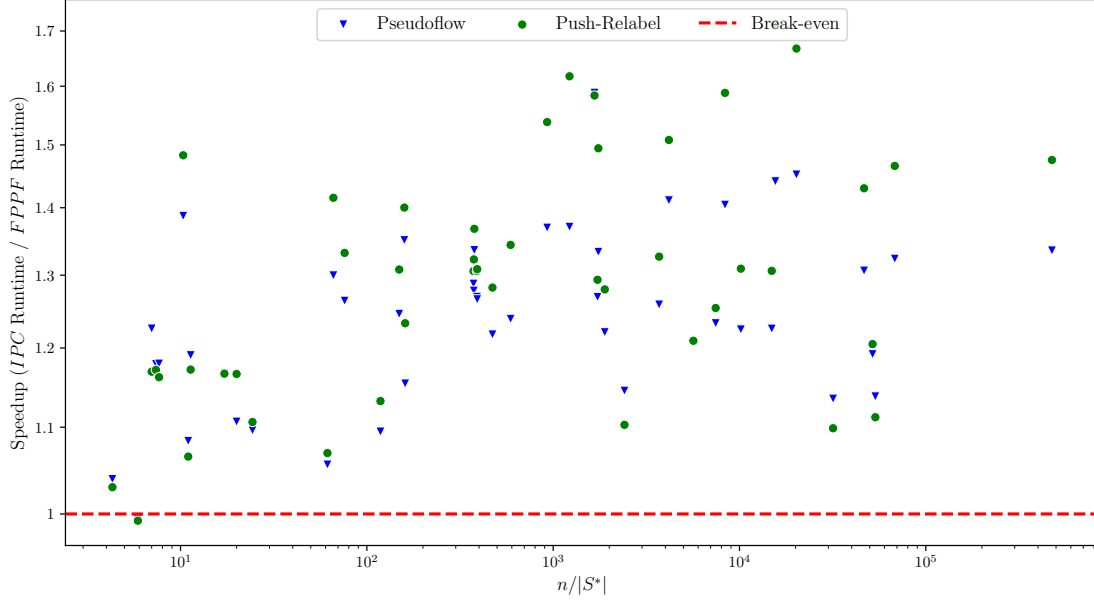


Figure 7.1.: Speedup of IPC in comparison to DSP-Intersect plotted against the ratio between the vertices in the whole graph and the vertices in the denset sub-graph. Calculated as $\frac{\text{runtime}(\text{DSP-Intersect})}{\text{runtime}(\text{IPC})}$. A speedup value greater than 1.0 indicates that IPC was faster than DSP-Intersect

	weight	min	10%	25%	mean	median	75%	90%	max	var
Excluded	unweighted	0.71	0.81	1.06	1.14	1.13	1.31	1.40	1.42	0.04
Excluded	weighted	0.77	0.81	0.94	1.20	1.32	1.45	1.47	1.64	0.08
Included	unweighted	0.75	0.87	1.03	1.07	1.07	1.17	1.21	1.23	0.02
Included	weighted	0.84	0.88	0.93	1.09	1.12	1.23	1.24	1.31	0.02

Table 7.3.: Speedup of IPC_{PF} in comparison to IPC_{PR} , while excluding and including the time for the creation of the flow network. Divided into weighted and unweighted. Calculated as $\text{runtime}(\text{IPC}_{PF})/\text{runtime}(\text{IPC}_{PR})$.

creation of the flow network, the runtimes get closer, the speedup being in median now 1.07 for unweighted and 1.12 for weighted instances.

To understand when which version is faster, we plotted the speedup against several graph metrics:

- Number of vertices n
- Number of edges m

- Ration of edges to vertices m/n
- Graph density (only different to ratio of edges to vertices for weighted graphs)
- degree variance
- degree variance unweighted (calculated for the weighted instances as if the edges were unweighted)
- Graph density / optimal value
- (m/n) / optimal value
- Iterations performed by the algorithm

The resulting plot can be seen in Figure 7.2.

The most insightful metric was the ratio of **graph density to the optimal density** ($\frac{\text{Graph Density}}{\lambda^*}$). As seen in the plot, there is a clear trend: the smaller this ratio, the faster IPC_{PR} becomes relative to IPC_{PF} . Since the graph density is the starting density for the algorithm, a smaller value describes a higher distance of the start value to the optimal value. It seems that the greater this distance and the smaller the ratio the better PR performs in comparison to PF.

Since the optimal density is unknown before running the algorithm, a practical proxy for the *unweighted instances* is the **degree variance**. The plot indicates that as degree variance increases, the speedup of IPC_{PR} tends to rise. However, this metric is not a perfect predictor, as it fails to explain the behavior of instances such as **BlogCatalog**, which is accurately captured by the $\frac{\text{Graph Density}}{\lambda^*}$ ratio. PF runs faster only on the instances **BlogCatalog3**, **com-amazon**, and the three **roadNet** graphs. Excluding these cases, PR is faster on all other instances, achieving a speedup over PF ranging from 1.02 to 1.41, with a median of 1.22.

For weighted graphs, while the plot shows a similar trend for $\frac{\text{Graph Density}}{\lambda^*}$, we do not observe the same relationship with variance as in the unweighted cases. We found no pre-run metric that consistently explains the behavior of the weighted instances. The small sample size and the heterogeneous nature of the weighted graphs are likely major contributors to this diverse differences.

Still we noticed that on the instances **mouse_gene**, **human_gene1** and **human_gene2** PF was faster. It seems that for those *gene type* graphs PF is preferable.

7.4. Comparing DSP-FBA and IPC

7.4.1. Unweighted Instances

This analysis compares the speedup of IPC_{PR} and IPC_{PF} relative to **DSP-FBA** with **FPPF**. The initial hypothesis was that the performance of FPPF might be related to how close its starting upper bound u is to the optimal value. The graph on the right side of Figure 7.3 shows that a larger ratio of the upper bound to the optimal value correlates

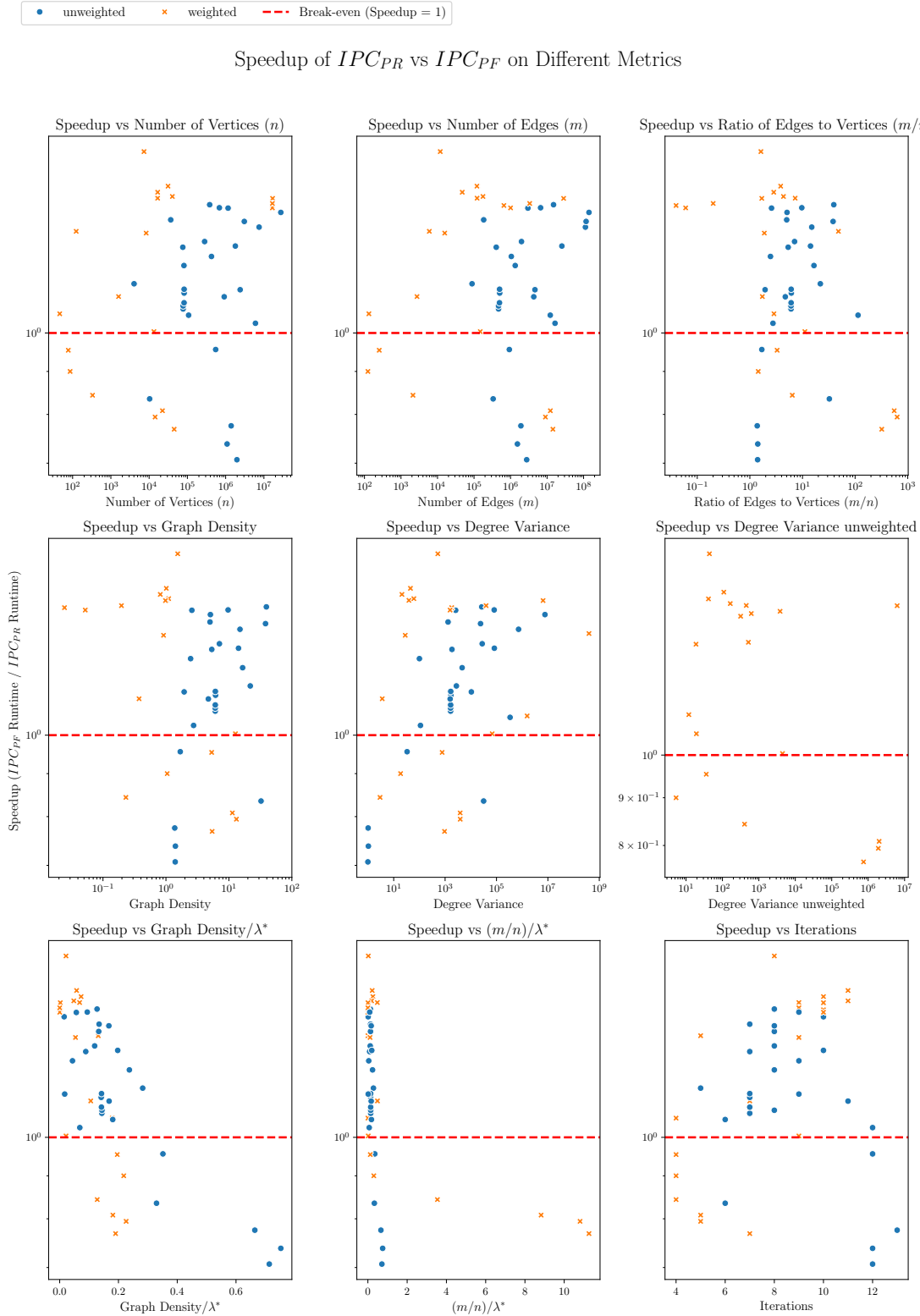


Figure 7.2.: Speedup of IPC using push-relabel (PR) compared to IPC using pseudoflow (PF) across various graph metrics. A speedup value greater than 1 indicates that PR was faster. Divided by weighted and unweighted instances.

with a slower performance of FPPF compared to IPC. This relationship is logical as the ratio indicates the search space that FPPF must traverse to find the optimal value.

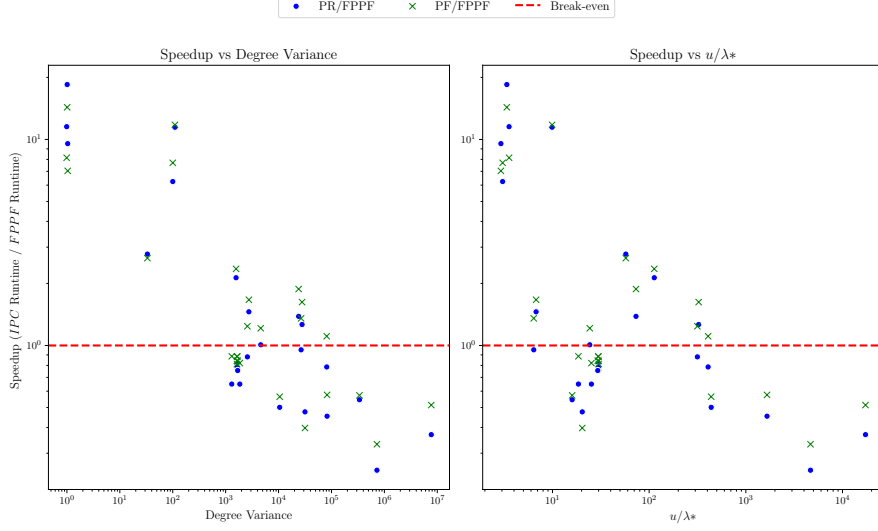


Figure 7.3.: Speedup of IPC using Push-Relabel and Pseudoflow compared to DSP-FBA using FPPF. A speedup value greater than 1.0 indicates that DSP-FBA using FPPF was faster.

Since the optimal value is not known in advance, we examined the variance of the vertex degrees. The graph shows an even stronger **inverse relationship** between the **variance of vertex degrees** and the speedup of FPPF over IPC. This makes sense because the upper bound u is defined as the maximum degree. A larger variance in degrees often results in a higher maximum degree, leading to a larger upper bound and, consequently, a larger search space for FPPF.

This observation is strengthened by the previous finding that as **variance decreases**, IPC_{PF} performs better than IPC_{PR} . Since FPPF is based on the pseudoflow algorithm, this correlation is logical.

Therefore, it seems logical to use DSP-FBA for instances with small variance and IPC_{PR} for those with large variance.

This raises the question of where to place the boundary between small and large variance. Revisiting Figure 7.3, three distinct groups emerge:

1. **Small** (variance < 1250): FPPF is consistently the fastest for these instances.
2. **Large** (variance $> 30,000$): IPC_{PR} is consistently the fastest for these instances.
3. **Medium** ($1250 \leq \text{variance} \leq 30,000$): Performance varies; sometimes FPPF is faster and sometimes IPC, but the difference is generally minor. On these instances IPC_{PR} is *always* faster than IPC_{PF} .

If we now look at the speedup between IPC_{PR} and FPPF on just those medium instances only we get the descriptive statistics seen in the top row of Table 7.4.

	min	10%	25%	mean	median	75%	90%	max	var
Excluded	0.50	0.65	0.76	0.99	0.81	1.14	1.43	2.13	0.18
Included	0.66	0.79	0.86	0.97	0.89	1.06	1.19	1.51	0.04

Table 7.4.: Speedup of IPC_{PR} in comparison to DSP-FBA with FPPF on the unweighted instances with medium variance. Divided between including and excluding the creation time for the flow network. Calculated as $\frac{runtime(IPC_{PR})}{runtime(DSP-FBA_{FPPF})}$. Value greater 1 means DSP-FBA is faster

On medium-sized instances, both algorithms show comparable performance, with the maximum observed speedup reaching 2.13. Overall, the IPC_{PR} algorithm is on average slightly faster than FPPF (mean speedup: 0.99). The median speedup of 0.81 indicates that FPPF is more often slower, but achieves more pronounced speedups in the cases where it outperforms IPC_{PR} .

The Creation of the Flow Network The comparison could always exclude the time required to construct the flow network, since this process is nearly identical across all algorithms. The construction is exactly the same for all IPC variants and DSP-Intersect, and almost identical for DSP-FBA, differing only in edge rotations and the flow function. Thus, we evaluated the algorithms' runtimes without considering this flow network creation time. Nevertheless, this construction can constitute a substantial portion of the overall runtime, and thus should be included when assessing actual speedups.

Most interesting is the comparison on the medium instances described in Section 7.4.1, since for these instances neither algorithm is consistently faster. Looking at the descriptive statistics of the speedup, now including the flow network construction on the medium instances in the bottom row of Table 7.4, the algorithms appear even more alike. In fact, the maximum observed performance ratio is only 1.51, with the median now being 0.89 and the mean 0.97. This minor discrepancy suggests that choosing IPC_{PR} for medium is the slightly better choice, but not very significant. A more strategic approach might be to divide the medium instances between the two algorithms. This would reduce the potential for error at the transitional boundary between small and large variance, yielding a more robust analysis.

7.4.2. Weighted Instances

While in Section 7.3 we noted that, for weighted instances, no consistent pattern emerges indicating whether IPC_{PR} or IPC_{PF} is faster, we observed that whenever IPC_{PF} outperforms IPC_{PR} , it also outperforms DSP-FBA with FPPF. These cases include the *gene-type* instances as well as three additional datasets.

IPC_{PR} was the fastest only on three graphs with extremely high degree variance, GD97_b, twitter-follows, and Journals, which have the highest variance by a large margin. Although the data shows no definitive trend, these exceptions for large variance suggest that degree variance again appears to influence relative speedups. Nevertheless, DSP-FBA with FPPF is the fastest on most weighted instances.

As shown in Table 7.5, especially when including the time required for constructing the flow network, the relative performance of all algorithms is similar. FPPF is at worst only 0.52 and 0.60 times as fast as IPC_{PF} and IPC_{PR} , respectively, while achieving a maximum speedup of 2.13 and 1.83 to IPC_{PF} and IPC_{PR} .

	flow network time	min	10%	25%	mean	median	75%	90%	max	var
PF/FPPF	Excluded	0.36	0.41	0.56	2.43	2.04	3.78	5.01	6.58	4.30
PR/FPPF	Excluded	0.43	0.51	0.55	1.81	1.74	2.56	3.31	5.01	1.89
PF/FPPF	Included	0.52	0.54	0.68	1.28	1.37	1.81	1.92	2.13	0.35
PR/FPPF	Included	0.60	0.62	0.66	1.13	1.29	1.46	1.58	1.83	0.18

Table 7.5.: Speedup of FPPF in comparison to IPC_{PR} and IPC_{PF} excluding and including the time needed for the construction of the flow network on weighted instances.

7.5. Iterations performed by Greedy++

Table 7.6 shows the iterations *Greedy++* performed to achieve our desired percentage of the optimal Value on the unweighted Instances.

As Boob et al. (2020) observed, and we can also see in our experiments, 90 Percent of the optimum was always achieved within 3 iterations. We again see the three **roadnet**-instances as outlier, as well as **com-amazon**, every other instance only needed 1 iteration to find 90 percent of the optimum. Naturally, more iterations are required to get closer to the optimal value. While Greedy++ required a relatively small number of iterations to find the true optimum on many instances, the number could become very high in some cases. The distribution of iterations needed to achieve the optimum is as follows: Median 2, Mean 18.72, 75% 10, 90% 52 and maximum 207 with a variance of 1950.127.

For the weighted instances, we do not provide a similar table, since *Greedy++* reached 90% of the optimum in a single iteration on all instances, and 100% of the optimum in one iteration for almost all cases; only **astro-ph** required a second iteration to reach full optimum. This demonstrates that, on weighted instances, both Greedy++ and the simple Greedy algorithm perform exceptionally well.

7.6. Comparing Greedy++ to the Exact Algorithms

For the peeling algorithms, there is a significant difference in runtime between unweighted and weighted instances. As discussed in Section 4.2, in the weighted case the vertex with

instance	0.8	0.9	0.95	0.99	1.0
BlogCatalog3 [2]	1	1	1	1	1
Ego-twitter	1	1	1	2	21
actor-collaboration [1]	1	1	1	1	2
cit-Patents	1	1	1	1	1
com-amazon	2	3	5	20	64
com-dblp	1	1	1	1	2
com-youtube	1	1	1	1	79
ego-gplus	1	1	1	1	2
email-Enron	1	1	1	1	2
facebook	1	1	1	1	1
livejournal-groupmemberships [1]	1	1	1	1	-
orkut-links [1]	1	1	1	1	-
roadNet-CA	1	2	6	10	25
roadNet-PA	1	3	6	36	207
roadNet-TX	1	2	4	4	4
soc-Epinions1	1	1	1	1	1
soc-Slashdot0811	1	1	1	1	1
soc-Slashdot0902	1	1	1	1	1
soc-sign-Slashdot081106	1	1	1	1	1
soc-sign-Slashdot090216	1	1	1	1	1
soc-sign-Slashdot090221	1	1	1	1	1
trackers [1]	1	1	1	1	3
web-BerkStan	1	1	1	1	1
web-Google	1	1	1	3	10
web-Stanford	1	1	1	1	34
wiki-Talk	1	1	1	1	1
wiki-topcats	1	1	1	2	2

Table 7.6.: Iterations performed by Greedy++ until it achieved the desired percentage of the optimum for the unweighted instances.

the smallest degree cannot be found using degree lists. Instead, the implementation must rely on a priority queue or a balanced binary search tree, as in the approach of Boob et al. (2020). They reported that this implementation is 6 to 10 times slower than the unweighted version. Therefore, we separate the comparison into unweighted and weighted instances.

7.6.1. Comparing Greedy++ to the Exact Algorithms on Unweighted Instances

Since Greedy++ is typically used to find a *good approximation* rather than the exact optimal value, for the unweighted instances we compare our exact algorithms against Greedy++ when it is run for **3 iterations**. This ensures that we are comparing our algorithms with a version of Greedy++ that reliably achieves at least 90% of the optimum.

Based on the performance analysis in the previous sections, we will compare the most effective algorithms. This comparison focuses on Greedy++ with 3 iterations (G-3) in comparison with DSP-FBA with FPPF for the *small* instances, IPC_{PR} for the *large* and both for the *medium*, as we explained in Section 7.4]

The speedup is determined by dividing the exact algorithm's runtime by the runtime of Greedy++ while running 3 iterations. Obviously this runtime now includes the time needed for the creation of the flow network. The descriptive statistic of these speedups divided by small, medium and large can be seen in Table 7.7.

type	algorithm	min	10%	25%	mean	median	75%	90%	max	var
Small	FPPF	0.77	0.97	1.20	1.32	1.37	1.54	1.62	1.68	0.11
Medium	FPPF	0.84	1.29	1.75	2.58	2.66	3.49	3.60	3.65	1.00
Medium	IPC _{PR}	1.02	1.35	1.88	2.36	2.76	2.96	3.04	3.06	0.52
Large	IPC _{PR}	0.88	0.95	1.04	1.59	1.21	2.20	2.61	2.74	0.65

Table 7.7.: Descriptive statistics of the speedup of Greedy++ with 3 iterations (G-3) compared to exact algorithms on unweighted instances. Divided by variance of the degree into small (< 1250), large ($> 30\,000$) and medium. The speedup is calculated as $\frac{\text{runtime}(\text{exact})}{\text{runtime}(\text{G-3})}$. A value > 1 indicates that Greedy++ with 3 iterations is faster than the exact algorithm. A value < 1 indicates G-3 is slower.

We observe that the maximum speedup achieved by **G-3** is:

- **1.68** compared to **FPPF** in the small group,
- **2.74** compared to **IPC_{PR}** in the large group, and
- in the medium group, up to **3.65** against **FPPF** and **3.06** against **IPC_{PR}**.

On average, the algorithms are closer to each other:

- **Small group:** mean **1.32**, median **1.37**,
- **Large group:** mean **1.59**, median **1.21**,
- **Medium group:** mean **2.58** (FPPF), **2.36** (IPC_{PR}); median **2.66** (FPPF), **2.76** (IPC_{PR}).

This indicates that by selecting **IPC_{PR}** for the medium and large groups and **FPPF** for the small group, we consistently achieve the best performance. In particular, we reach the optimal value in at most **3.06** times the runtime of three iterations of **Greedy++**, which itself does not guarantee (though in our experiments always achieved) 90% of the optimum.

7.6.2. Comparing Greedy++ to the Exact Algorithms on Weighted Instances

Using the same logic as for the unweighted instances, we compare against the version of Greedy++ that reliably achieves at least 90% (G-90) of the optimum which actually needs only one iteration .

Because no single exact algorithm emerged as consistently best, we evaluate DSP-FBA with FPPF, IPC_{PR} , and IPC_{PF} against G-90. Descriptive statistics for the weighted instances are shown in Table 7.8.

	min	10%	25%	mean	median	75%	90%	max	var
$FPPF$	0.57	0.62	0.74	1.26	1.07	1.57	2.20	2.95	0.46
IPC_{PR}	0.87	0.89	0.92	1.21	1.11	1.34	1.78	2.00	0.12
IPC_{PF}	0.80	0.81	1.05	1.31	1.27	1.66	1.80	1.95	0.14

Table 7.8.: Descriptive statistics of the speedup of Greedy++ with a 90% target (G-90, one iteration) compared to exact algorithms on *weighted* instances. Speedup is calculated as $\frac{\text{runtime}(\text{exact})}{\text{runtime}(\text{G-90})}$. A value > 1 indicates G-90 is faster, a value < 1 indicates it is slower.

The median speedups are modest: FPPF 1.07, IPC_{PR} 1.11, and IPC_{PF} 1.27. This shows that, on typical weighted instances, the exact algorithms are only *slightly slower* than one iteration of Greedy++. The maximum observed speedup is 2.95 against FPPF, 2.00 against IPC_{PR} , and 1.95 against IPC_{PF} . In fact, for most instances the FPPF speedup stays below 2, with the exceptions being the three smallest graphs (fewer than 300 edges). These are also cases where G-90 achieves the largest advantage over IPC. The higher relative speedup here can be attributed to the overhead of the more complex flow-based algorithms compared with the simpler Greedy approach.

Excluding these three smallest instances yields the revised statistics in Table 7.9.

	min	10%	25%	mean	median	75%	90%	max	var
FPPF	0.57	0.61	0.66	1.04	0.87	1.40	1.58	1.96	0.18
IPC_{PR}	0.87	0.88	0.92	1.10	1.02	1.21	1.36	1.66	0.05
IPC_{PF}	0.80	0.81	1.00	1.23	1.09	1.48	1.72	1.80	0.11

Table 7.9.: Descriptive statistics of the speedup of Greedy++ with 90% target (G-90) (one iterations) compared to exact algorithms on the weighted instances *excluding the smallest instances with less than 300 edges*. The speedup is calculated as $\frac{\text{runtime}(\text{exact})}{\text{runtime}(\text{G-90})}$. A value > 1 indicates that G-90 is faster than the exact algorithm. A value < 1 indicates it is slower.

With these smallest graphs removed, DSP-FBA with FPPF is, on median, slightly faster than G-90 and only about 4% slower in mean, with a maximum slowdown factor of 1.96. Both IPC variants show slightly lower maximum slowdowns (1.66 for IPC_{PR} and 1.80 for IPC_{PF}) while maintaining slightly higher medians (1.02 and 1.09, respectively) and means (1.10 and 1.23, respectively).

Conclusion Across weighted instances, Greedy++ with a 90% target, which runs just one iteration, provides at most a modest runtime advantage. After excluding graphs

with fewer than 300 edges, the flow-based exact methods are on par with or only slightly slower than Greedy++, indicating that exact algorithms remain competitive for practical instance sizes.

8. Graph Pruning

Since graph pruning is carried out by running one iteration of Charikars greedy algorithm (Charikar 2000) before applying the evaluated algorithm, the effect strongly depends on whether the instance is weighted or unweighted.

8.1. Size Reduction

Let us first consider the reduction in graph size that can be achieved through pruning. Descriptive statistics of this reduction are summarized in Table 8.1.

	min	10%	25%	mean	median	75%	90%	max	var
n/n^*	1.09	2.79	7.37	5 411.20	32.23	142.33	5 068.00	113 172.55	401 332 837.43
m/m^*	1.00	1.77	2.91	59.64	7.68	26.16	84.40	1 370.72	42 283.42

Table 8.1.: Descriptive statistics of the size of the graph after pruning. n and m describes the original size n^*, m^* the size after pruning.

We observe that pruning can substantially reduce the size of the graph. While the minimum reduction is relatively modest (unweighted $n/n^* = 1.24$, $m/m^* = 1.15$, weighted $n/n^* = 1.09$, $m/m^* = 1.00$), the median shows that graphs are typically shrunk by a factor of about 32.23 in vertices and 7.68 in edges. In extreme cases, pruning achieves reductions of more than 100 000 times in the number of vertices and more than 1 300 times in the number of edges. This large variance highlights that the effect of pruning strongly depends on the structure of the instance, but in most cases it yields a significant simplification of the input.

We observe the intuitive fact that instances with smaller variance in the degree distribution are less affected by pruning. The smallest reductions can be seen in the **roadNet** ($var = 0.99$ – 1.03) instances and in **com-amazon** ($var = 33.23$). For example, in the **roadNet** graphs the number of vertices is reduced only by a factor of about 1.24–1.27, and in **com-amazon** shrinking the vertex set by a factor of 3.25. A similar effect can be observed for the number of edges, where the reduction factor ranges between 1.15 and 1.17 for the **roadNet** graphs and only reaches 1.62 for **com-amazon**. Even the instances with the next higher variances, **com-dblp** and **cit-Patents**, although achieving vertex reductions of 7.06 and 153.58, still exhibit only modest edge reductions by factors of 2.46 and 2.75. On special case is the **netscience** instance, where only isolated vertices were removed and no edges ($n/n^* = 1.09$, $m/m^* = 1.00$).

These results illustrate the intuitive fact pruning is considerably more effective on graphs with larger degree variance, where more nodes with a small degree exists and can

be removed, whereas on graphs with nearly uniform degree distributions, such as the *roadNet* family, the effect is much more limited.

8.2. Variant P-1 of Pruning Unweighted Instances

As we have already observed, running *Greedy++* for three iterations is still, in most cases, faster than the exact algorithms. Therefore, we aimed to achieve an additional speedup by pruning the graph, as described in Section 5. We first compare against the Version P-1, the one that stops as soon as it can not remove a vertex.

8.2.1. Comparing the Speedup of Each Algorithm in Comparison to Unpruned Runtime

We first consider comparing the sole runtime of the algorithms in comparison to its own runtime before pruning, without taken in effect the time needed for the pruning itself (top rows in Table 8.2) and with it taken in effect (bottom rows in Table 8.2).

pruning time	algorithm	min	10%	25%	mean	median	75%	90%	max	var
Excluded	FPPF	1.21	2.05	7.69	77.06	15.14	25.88	138.32	800.62	35 673.54
Excluded	IPC _{PR}	0.83	1.72	7.83	130.58	16.49	40.87	165.03	1 500.68	128 525.46
Excluded	IPC _{PF}	0.83	2.05	10.19	130.38	20.27	63.77	216.46	1 353.07	106 881.32
Excluded	G-90	1.34	2.47	6.37	152.23	13.73	36.44	156.41	1 803.77	187 648.60
Excluded	G-100	1.10	2.64	5.26	160.21	14.84	40.15	181.39	1 956.85	212 435.79
Excluded	G-3	1.35	2.49	6.68	162.26	12.01	41.15	178.03	1 929.04	214 507.97
Included	FPPF	0.88	1.40	2.40	3.94	4.11	5.42	5.77	6.66	3.18
Included	IPC _{PR}	0.79	1.45	2.64	3.65	3.92	4.90	5.14	5.44	2.10
Included	IPC _{PF}	0.78	1.68	2.96	4.12	4.43	5.40	5.55	6.67	2.63
Included	G-90	0.69	0.80	0.88	0.94	0.92	0.96	1.00	1.69	0.03
Included	G-100	0.71	0.84	0.99	4.04	1.12	2.06	4.73	50.43	96.15
Included	G-3	0.86	1.19	1.96	2.09	2.12	2.41	2.69	3.09	0.33

Table 8.2.: Descriptive statistics of the speedup for each evaluated algorithm of the graph before vs after pruning. When including and excluding the time taken for the pruning. Calculated as $\frac{\text{runtime}(\text{original graph})}{\text{runtime}(\text{pruned graph})}$.

Interestingly, for IPC, pruning the graph beforehand on the *roadnet* instances actually results in worse performance compared to not applying pruning. For every other instance and every other algorithm evaluated here, excluding the pruntime the runtime after pruning is faster than before.

Taken into consideration the runtime needed for the pruning itself we get a similar picture, as seen on the bottom half of Table 8.2. We see a different result for each algorithm: The speedup gained from graph pruning varies among the tested algorithms, even when including the time required for pruning itself:

- FPPF: This algorithm shows a substantial performance improvement, with a median speedup of 4.11. Its speedup ranges from 0.88 to 6.67.

- IPC_{PR} : Similar to FPPF, pruning IPC_{PR} is also highly effective. It has a median speedup of 3.92 and its range is from 0.79 to 5.44.
- IPC_{PF} : This version profits even more than PR, which indicates again that for smaller degree variance PF is preferable, since the interval between start density and optimum is much closer together than before, a indicator for a better performance for PF, as seen in Section 7.3. It has a median speedup of 4.32 and ranges from 0.78 to 6.67.
- G-90: In most cases, pruning provides little to no benefit for this algorithm. It achieves a speedup only on a small number of instances, indicating that the time spent on pruning often outweighs the savings in subsequent computations.
- G-3: Pruning offers a moderate but consistent speedup. Its median speedup is 2.12, with observed values ranging from 0.87 to 3.10. G-3 benefits more than G-90 because all three iterations can contribute to the speedup, whereas G-90 typically has only a single profitable iteration.

The analysis confirms that for most graph instances, pruning is a worthwhile strategy for reducing overall runtime. The only exception appears to be the **roadnet** instances, where no speedup was observed across the board when including the pruning time. On these FPPF on the original instances performs best.

8.2.2. Ratio of Pruning Time vs Algorithm Time

This strong difference between the speedup of including and excluding the time needed for pruning showed can be explained by Table 8.3.

	min	10%	25%	mean	median	75%	90%	max	var
FPPF	0.50	0.91	1.73	22.61	1.89	9.12	23.75	305.65	4 214.82
IPC PR	0.22	0.86	2.10	34.76	3.47	15.90	44.91	428.51	9 093.60
IPC PF	0.27	0.99	2.50	30.52	3.75	16.48	51.40	319.59	5 560.76
G-90	1.18	4.40	12.20	183.16	14.19	53.78	170.99	1 934.58	243 977.89
G-100	0.06	1.08	2.78	116.03	12.98	20.96	141.74	1 595.35	121 853.29
G-3	1.27	1.67	4.47	68.63	5.23	20.66	65.47	727.98	33 870.41

Table 8.3.: Descriptive statistics of the ratio of pruning time to the sole algorithm runtime on the pruned graphs. Algorithm runtime not including pruning time. Calculates $\frac{\text{pruning time}}{\text{algorithm runtime}}$.

Pruning accounts for at least **22%** of the total runtime of the exact algorithms, with median ratios of **1.89** for FPPF, **3.47** for IPC_{PR} , and **3.75** for IPC_{PF} . For the peeling methods the effect is far stronger, reaching medians of **14.19** for G-90, **12.98** for G-100, and **5.23** for G-3. This shows that pruning can dominate the overall runtime, reducing the impact of the chosen algorithm.

Focusing on the exact algorithms, pruning frequently dwarfs their own runtimes. FPPF exhibits a median ratio of **1.89** but a mean of **22.61** and a maximum of **305.65**. IPC_{PR} is even more affected, with a **3.47** median, a **34.76** mean, and a **428.51** maximum. At least a quarter of instances already incur pruning costs **9–16 times** greater than the algorithm itself. These results make clear that, in practice, **pruning and not the algorithmic core often dictates total execution time**.

8.2.3. Comparing the DSP-FBA FPPF and IPC on Pruned Unweighted Instances

Although Section 8.2.2 shows that pruning consumes a substantial share of the total runtime and therefore reduces the influence of the chosen algorithm, we still see the same relationship between degree-variance and speedup when comparing FPPF and IPC.

All comparisons exclude the **roadnet** instances, for which pruning was already deemed unnecessary. Like before the pruning, we plot the speedup $\frac{\text{runtime}(\text{IPC})}{\text{runtime}(\text{FPPF})}$ against the Degree Variance. The trend remains the same (Figure 8.1): higher variance corresponds to a lower speedup of FPPF.

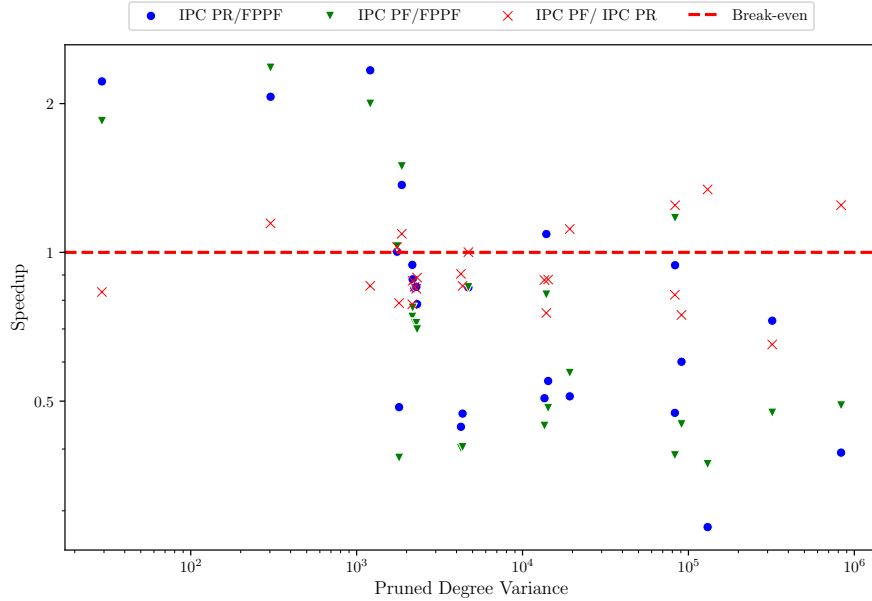


Figure 8.1.: Speedup of IPC compared to DSP-FBA using FPPF. Both including PF and PR. Also showing the speedup between IPC_{PR} and IPC_{PF} . Plotted in respect to the degree variance of the pruned graph. All speedups excluding the time taken for the pruning.

The general partitioning of instances based on degree variance still holds: FPPF is

preferred for the small instances, while IPC is generally faster for the large instances.

Since pruning the graph also changes the variance, the instance `web-google` now falls into the small partition, which is consistent with FPPF's observed speedup of 2.34 against IPC_{PR} and 2.00 against IPC_{PF} excluding the pruning time. These small instances are the only one where FPPF is significant faster, as we can see when we exclude them and summarize the descriptive statistics of the speedup between FPPF, IPC_{PR} and IPC_{PF} as seen in Table 8.4 on the remaining instances.

pruning time	algorithms	min	10%	25%	mean	median	75%	90%	max	var
Excluded	PR/FPPF	0.28	0.44	0.49	0.71	0.73	0.88	1.00	1.37	0.08
Excluded	PF/FPPF	0.37	0.39	0.45	0.66	0.57	0.77	1.03	1.49	0.09
Excluded	PF/PR	0.65	0.75	0.82	0.93	0.88	1.03	1.25	1.34	0.03
Included	PR/FPPF	0.72	0.79	0.92	0.93	0.95	0.98	1.00	1.00	0.01
Included	PF/FPPF	0.68	0.77	0.89	0.91	0.92	0.98	1.00	1.04	0.01
Included	PF/PR	0.90	0.94	0.95	0.98	0.99	1.00	1.02	1.06	0.00

Table 8.4.: Descriptive statistics of the speedups between both versions of IPC and FPPF. Only the pruned instances with a pruned degree variance greater than 1 250.

We already observed that IPC_{PF} benefits more from pruning than IPC_{PR} , and we now see that it more frequently emerges as the faster variant, as shown in Table 8.4. This aligns with our finding in Section 7.3: because all nodes are removed until the density no longer increases, the starting value lies much closer to the optimum and the graph is more uniform, making IPC_{PF} considerably more competitive.

Concretely, IPC_{PF} is in median **12% faster** than IPC_{PR} and **43% faster** than FPPF. On these instances IPC_{PR} is still **27% faster** than FPPF in median.

When including the time taken for pruning, both IPC variants are within **10%** of each other, with IPC_{PF} being slightly faster: about **1%** in median and **2%** in mean. The difference to FPPF decreases as well: **5%** in median for PR and **8%** for IPC_{PF} . IPC_{PR} still exhibits more consistent performance; while it is not the fastest on most instances, its worst-case runtime relative to FPPF is comparable, whereas IPC_{PF} can be up to **4%** slower.

Overall, for these pruned instances with moderate to high degree variance, using IPC is clearly the preferable choice, and the selection between PR or PF variants has minimal practical impact.

8.2.4. Variant 2 of Pruning Unweighted Instances

We introduced two pruning variants in Chapter 5. The thesis so far uses Pruning-1 (P-1) because its pruned-variance effect aligns with the observed algorithmic behavior. Pruning-2 (P-2) differs in executing the entire greedy procedure to obtain a 1/2-approximate solution. This approximation is then used as a tighter lower bound for IPC and DSP-Intersect, and when doubled and smaller than the usual bound as an upper bound for DSP-FBA.

For *Greedy++* this second variant is not sensible: the extra runtime of computing the full greedy solution yields no useful additional information. As we see in Figure 8.2 that, while P-2 always requires more time overall due to the additional greedy phase, this time decreases as pruning becomes more effective. This is a direct consequence of fewer remaining vertices to iterate over.

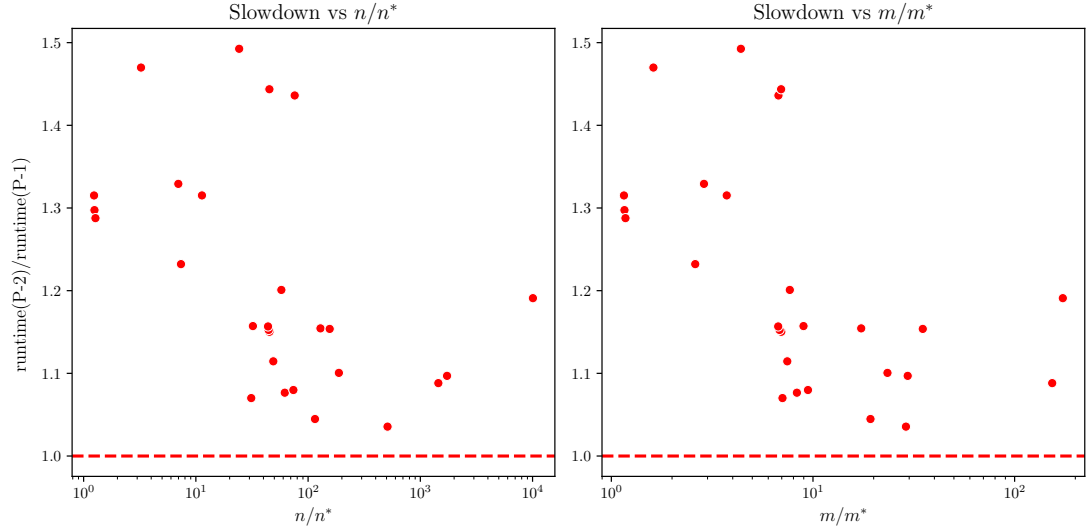


Figure 8.2.: Comparison between the runtime of P-1 and P-2

Due to the influence of the new bound on both exact algorithms, the speedups between IPC and FPPF show no consistent pattern. No structural metric explains the speedup because performance depends heavily on the quality of the Greedy solution.

Pruning Time	Method	min	10%	25%	mean	median	75%	90%	max	var
Excluded	FPPF	0.80	0.87	0.95	0.98	1.01	1.03	1.07	1.14	0.01
Excluded	IPC _{PR}	0.46	0.72	0.89	1.25	1.38	1.52	1.68	1.80	0.15
Excluded	IPC _{PF}	0.61	0.75	0.83	0.92	0.96	1.02	1.03	1.08	0.02
Included	FPPF	1.01	1.04	1.08	1.12	1.11	1.16	1.23	1.32	0.01
Included	IPC _{PR}	0.71	0.88	1.03	1.15	1.17	1.28	1.39	1.49	0.04
Included	IPC _{PF}	0.80	0.95	1.02	1.09	1.09	1.14	1.23	1.39	0.02

Table 8.5.: Descriptive statistics of the speedups between the algorithms when using the two pruning versions P-1 and P-2. Calculated as $\frac{P-2}{P-1}$, distinguishing between results that include and exclude the pruning time. Values greater than 1 indicate that the runtime using P-1 was faster than the runtime using P-2.

Examining the descriptive statistics of the speedups seen in Table 8.5 between the two pruning versions confirms that P-2 is generally slower than P-1 when the additional pruning time is included. The medians with pruning time are 1.11 for FPPF, 1.17 for

IPC_{PR} , and 1.09 for IPC_{PF} .

For FPPF, with P-2 it is never faster than P-1 when pruning time is counted. Even without pruning time, it is only slightly faster, with a mean of 0.98 and a median of 1.01. Because runtimes on pruned instances are very small, there are occasional outliers where FPPF is even slower with P-2, reflecting the variability at such small instance sizes.

For IPC_{PR} , the impact is both weakest and most variable. Excluding pruning time, the median indicates that PR with P-2 is 1.38 times slower than with P-1, yet in the best case it is more than twice as fast (0.46) and in the worst case nearly twice as slow (1.80).

IPC_{PF} is the most stable. Excluding pruning time, PF performs with P-2 performs slightly better, with a median of 0.96 and a mean of 0.92, but this advantage is offset by the longer pruning phase when the pruning time is included, as seen by the median and mean 1.09.

We are going forward with considering only P-1 in the comparison.

8.2.5. Final Comparison on Unweighted Instances: Greedy++ vs Exact Algorithms

We differentiate three groups of instances:

Instances where pruning provides no advantage For graphs with very uniform degree distributions and therefore very low degree variance pruning has little effect. The graph size is not significantly reduced and the pruning time outweighs any runtime savings. In these cases, using DSP-FBA with FPPF directly is preferable, as detailed earlier. Greedy++ also gains no benefit, so we compare FPPF against Greedy++ with three iterations (G-3) without pruning. Among our data, this applies only to the **roadNet** instances: G-3 is 1.57 times faster than FPPF on **roadNet-PA**, 1.27 on **roadNet-TX**, and 1.46 on **roadNet-CA**.

Instances where pruning helps and the variance is small For all other instances, pruning is worthwhile. When the resulting degree variance remains small, FPPF continues to be the best algorithm. Our experiments suggest a degree variance threshold of about 1250 for this category.

When both algorithms use P-1 on these instances, G-3 is only slightly faster than FPPF: 1.69 on **com-amazon**, 1.02 on **com-dblp**, and 1.13 on **web-google**. Compared with the original *unpruned* G-3, FPPF is much faster: the speedup of G-3 drops to 0.73 (**com-amazon**), 0.31 (**com-dblp**), and 0.40 (**web-google**).

Instances where pruning helps and the variance is large For the remaining pruned instances with larger degree variance, IPC becomes the preferred algorithm, while the choice between PR and PF has little practical impact. Descriptive statistics of the speedups appear in Table 8.6.

G-3	algo	min	10%	25%	mean	median	75%	90%	max	var
Pruned	IPC _{PR}	0.88	0.99	1.00	1.10	1.06	1.20	1.24	1.44	0.02
Pruned	IPC _{PF}	0.79	0.99	1.00	1.08	1.04	1.16	1.19	1.46	0.02
Original	IPC _{PR}	0.25	0.27	0.28	0.37	0.36	0.42	0.45	0.73	0.01
Original	IPC _{PF}	0.25	0.27	0.28	0.36	0.36	0.41	0.42	0.74	0.01

Table 8.6.: Descriptive statistics of the speedups between G-3 and IPC on Instances where pruning provides an advantage and the variance is large. IPC always uses P-1, while for G-3 it is differentiated between using P-1 (Pruned) and not using it (Original). Calculated as $\frac{exact}{G-3}$. Values greater than 1 indicate that G-3 was faster.

For these instances, G-3 is at most 1.46 times as fast as IPC when both use pruning; the median advantage is only 4% over IPC_{PF} and 6% over IPC_{PR}. On some instances IPC is faster than G-3. When comparing pruned IPC with the original unpruned G-3, the *exact algorithm is always faster*, with a median ratio of 0.36 and ranges from 0.25 to 0.74 (PR) and 0.73 (PF).

8.3. Pruning on Weighted Instances

The effectiveness of graph pruning on weighted instances is notably different from unweighted instances. The greedy algorithm, which underpins the pruning process, is generally slower on weighted graphs. As a result, pruning provides a smaller benefit to overall runtime. (For this analysis, we used Pruning-1 (P-1), as Pruning-2 was previously shown to be generally less effective for the exact algorithms on unweighted graphs, a finding that holds for the weighted case as well.

As shown in Table 8.7, the speedups are far less pronounced when including the pruning time. While the algorithms themselves run much faster on the pruned graphs (median speedup of 11.76 to 15.82 times without pruning time), this advantage is largely nullified by the time spent on pruning.

- **With Pruning Time Included:** The median speedup for most algorithms is **less than 1**, indicating that **pruning often makes the total process slower**. For example, the median speedup for FPPF is 0.76 and for IPC_{PR} is 0.92, showing that they are, in median, slower than their unpruned counterparts. Only IPC_{PF} shows a mean speedup slightly above 1 (1.06), making it the most stable performer with pruning. The only instances where a significant speedup for all three algorithms is observed are **twitter-follows**. Since this are by far the largest instances, it suggest that pruning should only be used on those.
- **FPPF:** On the gene type instance from all algorithm only FPPF benefits from pruning. These are the only one where FPPF benefits all. However, even on these, the unpruned IPC_{PF} algorithm remains the fastest choice.

pruning time	algorithm	min	10%	25%	mean	median	75%	90%	max	var
Excluded	FPPF	0.43	1.08	2.56	526.01	15.82	79.75	501.46	7 801.52	3 070 861.37
Excluded	IPC_{PR}	1.11	1.97	3.04	1 264.57	14.61	196.94	2 323.23	16 639.32	14 481 061.80
Excluded	IPC_{PF}	1.14	2.08	3.50	1 634.70	11.76	274.52	3 313.49	20 823.14	23 102 910.59
Excluded	G-90	1.03	2.32	3.06	650.17	15.11	97.89	316.43	10 411.31	5 459 825.21
Included	FPPF	0.34	0.55	0.61	0.83	0.76	0.96	1.25	1.72	0.11
Included	IPC_{PR}	0.71	0.81	0.83	0.95	0.92	1.05	1.18	1.34	0.03
Included	IPC_{PF}	0.74	0.77	0.86	1.06	0.99	1.25	1.47	1.65	0.08
Included	G-90	0.54	0.58	0.84	0.86	0.92	0.96	0.97	1.04	0.02

Table 8.7.: Descriptive statistics of the speedup for each evaluated algorithm of the graph after pruning, distinguishing between results that include and exclude the pruning time. Values greater than 1 indicate that the runtime after pruning was faster.

- **IPC:** The IPC algorithms are less affected by pruning than FPPF. IPC_{PR} shows a median speedup of 0.92, while IPC_{PF} is slightly faster with a median of 0.99. This suggests that the time taken for pruning is more likely to be a net loss for IPC_{PR} than for IPC_{PF} .
- **G-90:** The G-90 algorithm also shows a median speedup of 0.92, indicating that the pruning time typically outweighs the computational savings.

For weighted instances, **graph pruning is generally not a beneficial strategy**. The time consumed by the greedy pruning process frequently outweighs the time saved by running the algorithms on a smaller graph. This contrasts sharply with unweighted instances, where pruning is highly effective for graphs with high degree variance. The limited benefit from pruning on weighted graphs means that the best approach is often to run the algorithms directly on the original weighted graph.

The only exception arises for extremely large instances, where all exact algorithms benefit from pruning. This implies that for even larger weighted datasets, pruning could become highly advantageous, especially in scenarios where the original graph is too large to process efficiently without pruning.

9. Conclusion

This thesis presents a comprehensive empirical evaluation of exact and approximation algorithms for the Densest Subgraph Problem (DSP), challenging the conventional assumption that approximation methods are always preferable for large-scale graphs. Through systematic experimentation on 47 real-world graph instances, we demonstrate that modern exact algorithms can compete with and even occasionally outperform, established approximation techniques.

We introduced and evaluated two novel exact algorithms: **DSP-FBA**, which leverages first-breakpoint parametric flow techniques, and **DSP-Intersect**, which uses an intersection-based approach with restartable maximum flow similar to Hochbaum (2024) Incremental Parametric Cut procedure (IPC). Additionally, we provided the first comprehensive evaluation IPC for the DSP, given that their own empirical study is not publicly available. All exact algorithms exploit the properties of monotone parametric minimum cuts to avoid repeated static maximum flow computations, resulting in significant performance improvements over traditional approaches.

Our comparative analysis shows that **IPC consistently outperforms DSP-Intersect** when using identical underlying flow algorithms, primarily due to IPC’s more efficient parameter value calculation, particularly as the densest subgraph component shrinks.

Among the first breakpoint flow algorithms, **DSP-FBA using parametric pseudo flow (PPF)** demonstrates superior performance compared to parametric breadth-first search (PBFS), with DSP-FBA using PPF achieving speedups of 1.25 to 5.18 over the PBFS version.

A key finding contradicts prior claims that exact algorithms are magnitudes slower than approximation methods. On unweighted graphs, exact algorithms require at most 3.06 the runtime of Greedy++ with three iterations, making them viable when optimal solutions are required. This small performance gap is minimal considering the guarantee of optimality offered by exact methods. We identified **degree variance** as the primary factor determining algorithm performance on unweighted graphs. This leads to selection guidelines: **DSP-FBA with FPPF** excels on low-variance graphs ($< 1\,250$), **IPC with Push-Relabel** performs best on high-variance instances ($> 30\,000$), and both approaches are comparable on medium-variance graphs. For weighted instances, we could not identify a trend when which exact algorithm performs best, but DSP-FBA with FPPF generally emerges as the fastest algorithm.

Our experiments further demonstrate that the effectiveness of graph pruning strongly depends on both the graph structure and edge weight characteristics.

For unweighted graphs with high degree variance, pruning is highly beneficial: removing low-degree nodes can reduce the number of vertices by up to $100\,000\times$ and the number of edges by over $1\,300\times$. This substantially simplifies the graph and accelerates exact

algorithms. Conversely, for low-variance graphs such as the roadNet instances, pruning provides minimal reduction, and the time spent pruning outweighs any computational savings.

When pruning is effective, the variance of the resulting smaller instances changes. For these small instances, DSP-FBA with FPPF again emerges as the best exact algorithm, while IPC performs better on high-variance instances after pruning. Even Greedy++ benefits from pruning when running for three iterations, as each iteration profits from the reduced runtime. Comparing Greedy++ to the exact algorithm while distinguishing between small and large instances, the maximum remaining speedup is only 1.46, with the median being just 4% faster than IPC_{PF} and 6% faster than IPC_{PR} . Compared to the unpruned version of Greedy++, pruned exact algorithms always run faster, in median more than 2 times as fast. After pruning the choice between PR and PF shows nearly no significant difference in performance.

For weighted graphs, pruning is generally ineffective. The greedy algorithm used for pruning runs more slowly due to the underlying data structures, and the total runtime usually increases when pruning is applied. Median speedups drop below 1 when pruning time is included, suggesting that algorithms should typically be applied directly to the original weighted graph. Pruning only proved effective on the largest instance, indicating that it is useful for weighted graphs only when they are extremely large.

From these observations, several practical guidelines emerge:

1. Do not apply pruning on unweighted graphs with very low degree variance. Instead, use DSP-FBA with FPPF directly.
2. For unweighted graphs, choose IPC_{PR} when the pruned degree variance is high and DSP-FBA with FPPF when it is low. We observed a threshold around 1 250.
3. Pruning on weighted instances appears beneficial only for the largest graphs. Therefore, avoid pruning on most weighted instances unless the graphs are extremely large or your computer cannot handle them. On weighted graphs DSP-FBA with FPPF was generally the fastest.

These findings carry broad practical significance. Practitioners should not default to approximation methods, instead algorithm choice should reflect graph characteristics and solution requirements. When precision is critical, exact algorithms offer a feasible path to optimal solutions within reasonable computational limits. The implications extend beyond DSP to other combinatorial optimization problems expressible as parametric flow. This work highlights the promise of modern exact algorithms in real-world contexts and provides a framework for evaluating algorithms across both theoretical complexity and empirical performance. Future research may extend this analysis to additional graph families, develop adaptive algorithms that switch between exact and approximate methods in real time, and explore hybrid approaches that blend both paradigms. It could also investigate whether partial pruning, such as removing only a subset of low-degree nodes, improves runtime or develop efficient techniques for computing stronger initial bounds for exact algorithms.

A. Absolute Runtimes

Instance	n	m	FBA PPF	FBA PBFS	Int PR	Int PF	IPC PR	IPC PF	G-90
Sandi_authors	86	124	120	420	117	112	110	100	55
GD97_b	46	132	118	402	79	84	73	78	40
lesmis	77	254	181	415	142	148	133	124	75
USAir97	332	2 126	1 105	1 550	694	640	662	602	561
netscience	1 589	2 742	1 158	1 702	2 033	2 112	1 687	1 800	1 016
Journals	124	5 972	1 437	1 801	1 357	1 495	1 208	1 310	1 219
geom	7 343	11 898	4 563	7 265	9 499	11 871	7 228	9 445	5 612
hep-th	8 361	15 751	5 199	7 888	12 109	13 978	9 506	11 094	7 867
cond-mat	16 726	47 594	15 003	20 983	29 865	36 677	23 722	28 585	23 392
cond-mat-2003	31 163	120 029	36 673	51 215	72 188	83 368	54 298	67 320	62 088
astro-ph	16 706	121 251	29 887	43 615	49 588	57 823	41 719	48 331	48 374
Reuters911	13 332	148 038	39 929	57 341	60 877	61 208	47 546	47 335	52 174
cond-mat-2005	40 421	175 693	53 611	74 882	109 082	127 463	83 200	100 349	93 865
twitter-quotes	16 627 579	657 975	5 958 600	8 864 736	10 717 544	12 694 741	8 467 052	10 492 252	7 118 611
twitter-replies	16 636 365	1 002 496	6 108 587	9 163 693	11 285 245	13 335 820	8 902 036	10 973 794	6 048 969
twitter-retweets	16 647 272	3 342 154	6 958 722	10 572 903	11 963 963	14 274 784	9 654 173	11 938 624	7 883 316
human_gene2	14 340	9 027 024	4 726 760	8 081 994	2 963 000	2 606 323	2 922 689	2 551 981	3 174 771
human_gene1	22 283	12 323 680	6 393 586	10 741 537	4 170 427	3 677 530	4 209 046	3 698 835	4 555 695
mouse_gene	45 101	14 461 095	10 517 849	18 912 420	6 830 028	5 748 123	6 521 974	5 459 020	6 736 625
twitter-follows	16 853 360	28 395 679	31 822 535	0	25 347 530	29 723 423	20 776 476	25 966 873	19 285 856

Table A.1.: Overview of absolute runtimes on weighted instances on original unpruned graphs. Time in μ s.

Instance	n	m	FBA PPF	FBA PBFS	Int PR	Int PF	IPC PR	IPC PF	G-90	P-1
Sandi_authors	86	124	356	186	142	138	136	134	102	76
GD97_b	46	132	116	140	78	76	80	78	68	66
lesmis	77	254	417	415	163	154	162	151	125	101
USAir97	332	2 126	1 169	1 164	633	791	825	776	664	493
netscience	1 589	2 742	1 874	2 294	2 700	2 760	2 380	2 442	1 848	858
Journals	124	5 972	1 807	1 767	1 699	1 694	1 455	1 689	1 445	1 429
geom	7 343	11 898	6 155	6 458	6 123	6 121	6 117	6 110	6 093	6 052
hep-th	8 361	15 751	8 474	8 634	9 362	9 380	9 134	9 156	8 940	7 998
cond-mat	16 726	47 594	24 190	24 571	25 848	25 905	25 439	25 514	25 186	23 466
cond-mat-2003	31 163	120 029	63 477	63 923	65 636	65 668	65 117	65 112	64 942	62 574
astro-ph	16 706	121 251	48 732	49 261	50 162	50 245	49 786	49 832	50 270	47 360
Reuters911	13 332	148 038	53 634	53 690	53 582	53 581	53 580	53 571	53 554	53 527
cond-mat-2005	40 421	175 693	95 508	96 018	98 390	98 312	97 720	97 622	97 620	94 464
twitter-quotes	16 627 579	657 975	7 726 801	7 730 893	7 724 926	7 724 737	7 724 892	7 724 705	7 791 673	7 723 203
twitter-replies	16 636 365	1 002 496	6 637 834	6 637 804	6 637 594	6 637 586	6 637 586	6 637 578	6 637 632	6 637 050
twitter-retweets	16 647 272	3 342 154	8 215 004	8 196 758	8 177 642	8 176 906	8 177 582	8 176 922	8 255 052	8 172 814
human_gene2	14 340	9 027 024	3 852 936	4 861 810	2 995 294	2 759 994	2 975 288	2 762 054	3 410 672	2 061 028
human_gene1	22 283	12 323 680	5 140 578	6 622 204	4 245 950	3 846 469	4 383 357	3 870 515	4 778 995	3 070 534
mouse_gene	45 101	14 461 095	8 227 221	11 529 812	7 105 147	6 341 567	7 062 159	6 294 933	6 872 833	4 530 116
twitter-follows	16 853 360	28 395 679	18 483 528	18 648 608	18 428 674	18 425 198	18 428 698	18 425 552	18 541 654	18 382 952

Table A.2.: Overview of absolute runtimes on weighted instances on pruned graphs. Time in μ s.

Instance	n	m	FBA PPF	FBA PBFS	Int PR	Int PF	IPC PR	IPC PF	G-3
facebook	4039	88 234	18 033	23 936	22 278	23 212	20 884	23 065	7 577
email-Enron	36 692	183 831	88 388	143 301	84 517	96 564	71 162	82 706	24 284
BlogCatalog3 [2]	10 313	333 983	160 313	272 377	110 737	100 483	100 759	92 371	36 799
soc-Epinions1	75 888	405 740	208 413	357 553	191 523	213 415	163 732	185 637	57 118
soc-sign-Slashdot081106	77 357	468 554	242 369	388 500	253 909	261 459	213 312	219 088	69 792
soc-Slashdot0811	77 360	469 180	240 781	382 904	242 342	247 394	207 927	213 174	68 085
soc-sign-Slashdot090216	81 871	497 672	257 133	416 969	266 648	275 513	228 363	237 039	76 327
soc-sign-Slashdot090221	82 144	500 481	259 894	418 714	265 379	278 255	226 576	242 431	77 465
soc-Slashdot0902	82 168	504 230	257 269	401 313	257 418	268 488	220 417	231 709	73 221
com-amazon	548 552	925 872	927 080	1 362 998	2 018 773	1 948 228	1 742 535	1 681 993	551 899
com-dblp	425 957	1 049 866	493 762	733 191	1 159 503	1 287 989	968 163	1 098 106	421 024
Ego-twitter	81 306	1 342 296	496 075	767 488	581 685	634 870	493 248	547 089	231 126
roadNet-PA	1 090 920	1 541 898	910 921	1 530 725	4 132 622	3 292 306	3 798 315	2 942 236	580 429
roadNet-TX	1 393 383	1 921 660	940 780	1 437 206	5 307 215	4 348 803	4 836 794	3 912 370	738 462
web-Stanford	281 904	1 992 636	1 061 038	1 577 449	1 385 637	1 576 011	1 197 668	1 382 781	567 942
roadNet-CA	1 971 281	2 766 607	1 578 878	2 618 383	8 029 847	6 128 988	7 394 702	5 518 217	1 079 325
com-youtube	1 157 828	2 987 624	2 599 189	4 161 616	2 857 466	3 379 310	2 403 121	2 938 564	1 589 451
web-Google	916 428	4 322 051	3 046 716	4 479 159	5 361 970	5 674 575	4 604 444	4 913 696	2 307 004
wiki-Talk	2 394 385	4 659 565	6 076 361	9 189 822	4 591 302	4 870 589	4 010 578	4 236 730	2 280 674
web-BerkStan	685 231	6 649 470	2 533 849	3 236 818	2 764 326	3 195 856	2 208 248	2 661 726	886 372
ego-gplus	107 614	12 238 285	6 614 523	12 548 612	4 671 700	4 776 889	4 386 691	4 510 332	4 957 266
actor-collaboration [1]	382 220	15 038 083	5 930 085	10 651 084	6 491 628	7 740 909	5 758 418	7 064 969	4 659 234
cit-Patents	6 009 555	16 518 947	10 215 084	15 530 578	30 746 846	31 080 188	26 962 759	27 392 256	13 224 426
wiki-topcats	1 791 489	25 444 207	27 486 534	37 703 677	18 382 146	21 029 636	15 629 037	18 257 172	14 200 113
livejournal-groupmemberships [1]	7 489 074	112 305 407	210 234 681	0	77 920 550	93 107 098	75 010 538	89 888 199	73 805 781
orkut-links [1]	3 072 442	117 184 899	65 533 420	0	86 564 253	104 489 176	79 698 869	98 066 232	78 259 280
trackers [1]	27 665 731	140 613 747	233 390 966	803 042 884	141 486 439	167 779 724	121 660 254	147 077 799	92 053 556

Table A.3.: Overview of absolute runtimes on unweighted instances on original unpruned graphs. Time in μ s.

Instance	n	m	FBA PPF	FBA PBFS	Int PR	Int PF	IPC PR	IPC PF	G-3	P-1
facebook	4039	88 234	7 790	9 889	7 996	7 983	7 804	7 923	5 431	3 074
email-Enron	36 692	183 831	16 442	27 178	13 082	12 406	13 076	12 406	11 704	9 892
BlogCatalog3 [2]	10 313	333 983	36 976	62 519	27 034	25 290	26 717	25 233	22 447	14 214
soc-Epinions1	75 888	405 740	39 874	61 662	31 472	30 746	31 332	30 682	29 646	24 518
soc-sign-Slashdot081106	77 357	468 554	45 200	59 756	45 938	42 468	44 288	41 016	34 876	29 064
soc-Slashdot0811	77 360	469 180	43 919	57 667	43 256	41 336	42 040	40 299	33 892	28 014
soc-sign-Slashdot090216	81 871	497 672	49 201	65 342	47 903	45 310	46 625	44 312	38 188	31 780
soc-sign-Slashdot090221	82 144	500 481	50 212	65 761	47 646	45 892	46 313	44 729	38 491	32 029
soc-Slashdot0902	82 168	504 230	45 730	59 066	44 486	42 394	43 312	41 328	35 948	29 950
com-amazon	548 552	925 872	536 173	745 171	1 074 845	944 231	972 491	838 536	318 149	177 898
com-dblp	425 957	1 049 866	179 114	188 882	202 974	207 159	197 172	202 240	175 533	162 153
Ego-twitter	81 306	1 342 296	121 587	149 768	118 989	118 237	117 069	117 094	103 780	91 635
roadNet-PA	1 090 920	1 541 898	992 213	1 501 564	4 286 692	3 642 455	4 032 519	3 362 188	671 306	245 203
roadNet-TX	1 393 383	1 921 660	1 069 988	1 482 129	6 502 527	3 653 494	6 151 749	3 298 038	840 938	311 630
web-Stanford	281 904	1 992 636	268 742	276 554	256 078	257 162	255 574	257 158	248 246	241 794
roadNet-CA	1 971 281	2 766 607	1 740 428	2 598 844	9 021 632	7 704 436	8 533 848	7 085 748	1 237 218	437 744
com-youtube	1 157 828	2 987 624	632 940	687 906	612 798	609 540	612 652	610 027	603 590	594 530
web-Google	916 428	4 322 051	1 227 477	1 535 134	1 982 943	1 828 334	1 828 361	1 676 653	1 088 084	777 528
wiki-Talk	2 394 385	4 659 565	977 416	1 032 494	959 666	957 234	959 156	956 888	954 824	940 386
web-BerkStan	685 231	6 649 470	463 366	493 086	473 962	498 240	456 882	482 768	392 050	351 472
ego-gplus	107 614	12 238 285	2 168 774	3 278 902	1 955 596	1 749 928	1 952 198	1 750 694	2 210 212	1 374 506
actor-collaboration [1]	382 220	15 038 083	1 875 051	2 290 747	1 760 233	1 715 959	1 765 020	1 723 160	1 705 795	1 599 492
cit-Patents	6 009 555	16 518 947	4 985 770	4 990 790	4 993 788	4 995 327	4 991 768	4 993 785	4 977 936	4 969 511
wiki-topcats	1 791 489	25 444 207	4 741 478	4 880 664	4 785 770	4 722 464	4 761 710	4 700 934	4 592 642	4 514 588
livejournal-groupmemberships [1]	7 489 074	112 305 407	36 583 622	23 891 876	28 849 980	30 096 024	28 884 438	30 112 206	30 524 530	23 891 876
orkut-links [1]	3 072 442	117 184 899	29 648 476	36 847 768	27 759 683	27 456 425	27 760 472	27 456 036	28 097 208	26 063 133
trackers [1]	27 665 731	140 613 747	35 030 730	35 023 972	34 820 766	34 846 604	34 820 286	34 847 915	34 786 936	34 739 216

Table A.4.: Overview of absolute runtimes on unweighted instances on pruned graphs. Time in μ s.

Bibliography

- Angriman, Eugenio, Alexander van der Grinten, Moritz von Looz, Henning Meyerhenke, Martin Nöllenburg, Maria Predari and Charilaos Tzovas (2019). ‘Guidelines for Experimental Algorithmics: A Case Study in Network Analysis’. In: *Algorithms* 12.7. ISSN: 1999-4893. DOI: 10.3390/a12070127.
- Arora, Sanjeev, Elad Hazan and Satyen Kale (1st May 2012). ‘The Multiplicative Weights Update Method: a Meta-Algorithm and Applications’. In: *Theory of Computing* 8. Number: 6 Publisher: Theory of Computing, pp. 121–164. DOI: 10.4086/toc.2012.v008a006.
- Beines, Arne, Michael Kaibel, Philip Mayer, Petra Mutzel and Jonas Sauer (2025). ‘A Simpler Approach for Monotone Parametric Minimum Cut: Finding the Breakpoints in Order’. In: *2025 Proceedings of the Symposium on Algorithm Engineering and Experiments (ALENEX)*. eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9781611978339.3>, pp. 29–41. DOI: 10.1137/1.9781611978339.3. URL: <https://epubs.siam.org/doi/abs/10.1137/1.9781611978339.3>.
- Boob, Digvijay, Yu Gao, Richard Peng, Saurabh Sawlani, Charalampos Tsourakakis, Di Wang and Junxing Wang (20th Apr. 2020). ‘Flowless: Extracting Densest Subgraphs Without Flow Computations’. In: *Proceedings of The Web Conference 2020*. WWW ’20: The Web Conference 2020. Taipei Taiwan: ACM, pp. 573–583. ISBN: 978-1-4503-7023-3. DOI: 10.1145/3366423.3380140.
- Charikar, Moses (2000). ‘Greedy Approximation Algorithms for Finding Dense Components in a Graph’. In: *Approximation Algorithms for Combinatorial Optimization*. Ed. by Klaus Jansen and Samir Khuller. Berlin, Heidelberg: Springer, pp. 84–95. ISBN: 978-3-540-44436-7. DOI: 10.1007/3-540-44436-X_10.
- Chekuri, Chandra, Kent Quanrud and Manuel R. Torres (Jan. 2022). ‘Densest Subgraph: Supermodularity, Iterative Peeling, and Flow’. In: *Proceedings of the 2022 Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*. Proceedings. Society for Industrial and Applied Mathematics, pp. 1531–1555. DOI: 10.1137/1.9781611977073.64.
- Davis, Timothy A. and Yifan Hu (Dec. 2011). ‘The University of Florida Sparse Matrix Collection’. In: *ACM Transactions on Mathematical Software* 38.1, pp. 1–25. DOI: 10.1145/2049662.2049663.
- Eisner, Mark J. and Dennis G. Severance (1st Oct. 1976). ‘Mathematical Techniques for Efficient Record Segmentation in Large Shared Databases’. In: *J. ACM* 23.4, pp. 619–635. ISSN: 0004-5411. DOI: 10.1145/321978.321982.
- Ford Jr, L. R. and D. R. Fulkerson (Jan. 1956). ‘Maximal Flow Through a Network’. In: *Canadian Journal of Mathematics* 8, pp. 399–404. ISSN: 0008-414X, 1496-4279. DOI: 10.4153/CJM-1956-045-5.

- Gallo, Giorgio, Michael D. Grigoriadis and Robert E. Tarjan (Feb. 1989). ‘A Fast Parametric Maximum Flow Algorithm and Applications’. In: *SIAM Journal on Computing* 18.1, pp. 30–55. ISSN: 0097-5397, 1095-7111. DOI: 10.1137/0218003.
- Goldberg, A. V. (1984). *Finding a Maximum Density Subgraph*. UCB/CSD-84-171.
- Goldberg, Andrew V., Sagi Hed, Haim Kaplan, Robert E. Tarjan and Renato F. Werneck (2011). ‘Maximum Flows by Incremental Breadth-First Search’. In: *Algorithms – ESA 2011*. Ed. by Camil Demetrescu and Magnús M. Halldórsson. Vol. 6942. Series Title: Lecture Notes in Computer Science. Berlin, Heidelberg: Springer Berlin Heidelberg, pp. 457–468. ISBN: 978-3-642-23718-8 978-3-642-23719-5. DOI: 10.1007/978-3-642-23719-5_39.
- Goldberg, Andrew V. and Robert E. Tarjan (1st Oct. 1988). ‘A new approach to the maximum-flow problem’. In: *J. ACM* 35.4, pp. 921–940. ISSN: 0004-5411. DOI: 10.1145/48014.61051.
- Hochbaum, Dorit (2024). ‘Flow Is Best, Fast and Scalable: The Incremental Parametric Cut for Maximum Density and Other Ratio Subgraph Problems.’ in: *Proceedings of the 16th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management*. 16th International Conference on Knowledge Discovery and Information Retrieval. Porto, Portugal: SCITEPRESS - Science and Technology Publications, pp. 275–282. ISBN: 978-989-758-716-0. DOI: 10.5220/0012917300003838.
- Hochbaum, Dorit S. (Aug. 2008). ‘The Pseudoflow Algorithm: A New Algorithm for the Maximum-Flow Problem’. In: *Operations Research* 56.4, pp. 992–1009. ISSN: 0030-364X, 1526-5463. DOI: 10.1287/opre.1080.0524.
- Kunegis, Jérôme (13th May 2013). ‘KONECT: the Koblenz network collection’. In: *Proceedings of the 22nd International Conference on World Wide Web*. WWW ’13 Companion. New York, NY, USA: Association for Computing Machinery, pp. 1343–1350. ISBN: 978-1-4503-2038-2. DOI: 10.1145/2487788.2488173.
- Lanciano, Tommaso, Atsushi Miyauchi, Adriano Fazzzone and Francesco Bonchi (31st Aug. 2024). ‘A Survey on the Densest Subgraph Problem and Its Variants’. In: *ACM Computing Surveys* 56.8, pp. 1–40. ISSN: 0360-0300, 1557-7341. DOI: 10.1145/3653298. arXiv: 2303.14467[cs].
- Leskovec, Jure and Andrej Krevl (June 2014). *SNAP Datasets: Stanford Large Network Dataset Collection*. URL: <http://snap.stanford.edu/data>.
- Picard, Jean-Claude and Maurice Queyranne (1982). ‘A network flow solution to some nonlinear 0-1 programming problems, with applications to graph theory’. In: *Networks* 12.2. _eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/net.3230120206>, pp. 141–159. ISSN: 1097-0037. DOI: 10.1002/net.3230120206.
- Sauer, Jonas (2025). ‘A Faster Pseudoflow-Based Algorithm for Monotone Parametric Minimum Cut’. Unpublished manuscript.
- Sotiropoulos, Konstantinos, John W. Byers, Polyvios Pratikakis and Charalampos E. Tsourakakis (Sept. 2019). ‘TwitterMancer: Predicting User Interactions on Twitter’. In: *2019 57th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*. 2019 57th Annual Allerton Conference on Communication, Control, and

- Computing (Allerton). Monticello, IL, USA: IEEE, pp. 973–980. ISBN: 978-1-7281-3151-1. DOI: 10.1109/ALLERTON.2019.8919702.
- Stark, Chris, Bobby-Joe Breitkreutz, Teresa Reguly, Lorrie Boucher, Ashton Breitkreutz and Mike Tyers (1st Jan. 2006). ‘BioGRID: a general repository for interaction datasets’. In: *Nucleic Acids Research* 34 (suppl_1), pp. D535–D539. ISSN: 0305-1048. DOI: 10.1093/nar/gkj109.
- Wu, Bo and Haiying Shen (1st Sept. 2017). ‘Exploiting Efficient Densest Subgraph Discovering Methods for Big Data’. In: *IEEE Transactions on Big Data* 3.3, pp. 334–348. ISSN: 2332-7790. DOI: 10.1109/TBDATA.2016.2625307.
- Zafarani, R. and H. Liu (2009). *Social Computing Data Repository at ASU*. URL: <https://datasets.syr.edu/> (visited on 30/08/2025).